Fast Simulation-Based Bayesian Estimation of Heterogeneous and Representative Agent Models

Cameron Fen*

September 26, 2023

Heterogeneous agent models are rapidly becoming the workhouse model of modern macro but present estimation challenges compared to their representative agent counterparts. This paper proposes a technique for the estimation of structural macroeconomic models using a simulation-based algorithm for maximum likelihood or Bayesian estimation, dubbed Sequential Neural Posterior Estimation (SNPE). The approach aims to simulate from the joint distribution of parameters and simulated data, then fits a conditional density estimator on the joint samples for the parameters and the data to get the posterior. This method has the following advantages: estimation is independent of and is uneffected by the choice of solution method or the inner workings of the model, its speed and accuracy compared to Bayesian posterior benchmark methods, and statistically efficient simulation-based estimation. For example, the posteriors estimated on the benchmark Smets and Wouters (2007) are higher quality and faster compared to Metropolis-Hastings. With HANK models, the algorithm sidesteps lossy dimensionality reduction when using Reiter's method and improves estimation speed of models solved via Winberry's method by 13x per iteration. An empirical application estimating a full information heterogeneous agent model with both aggregate time series and cross sectional microdata demonstrates the capability of this technique. A tutorial can be found at: `https://shorturl.at/pMQ18`

JEL Codes: E27, E37, E47, C45, C11, C15
Keywords: Neural Networks, Bayesian Inference, Dynamic Estimation, Simulation-Based Estimators

*Cameron Fen is PhD student at the University of Michigan, Ann Arbor, MI, 48104 (E-mail: camfen@umich.edu. Website: cameronfen.github.io.). The author thanks Xavier Bautista, Florian Gunsilius, David Childers, Eric Jang, John Leahy, and Toni Whited and for helpful feedback and Alisdair McKay, Toni Whited, Emil Lakkis, Mohammed Ait Lahcen, and Jeppe Druedahl for code. All errors are my own.

I. Introduction

Despite the estimation challenges, the field in macro is moving towards the use of heterogeneous agent models, due to their increased realism[1]. I introduce a new machine approach, Sequential Neural Posterior Estimation (SNPE), that can estimate any black box method via simulations in a fast, accurate, and efficient MLE or Bayesian manner. Some problems with heterogeneous agent model estimation include a high dimensional state space, matching both macro and micro data, and reproducing oddly shape, multimodal, and unidentified posteriors. SNPE uses advances in machine learning conditional density estimator to handle the curse of dimensionality, matches real data with simulated data to avoid difficult likelihood functions for micro data, and, in the Bayesian setting, produces fast, accurate, and reproducible posteriors compared to benchmark methods. Without much modification, SNPE can estimate models solved via projection, perturbation, value function iteration, Reiter's method (Reiter, 2009), Winberry's method (Winberry, 2018), time iteration (Rendahl, 2017), the sequence space Jacobian approach (Auclert et al., 2021*a*), and more.

The objective of Bayesian estimation is to derive the posterior distribution, using Bayes' rule to derive the optimal distribution of the parameters after seeing the data. Bayes' rule derives the posterior with the likelihood weighted average of the prior parameter distribution, $P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$, where θ are the parameters and X is the simulated data. Since the method is likelihood free, one does not have access to the likelihood, $P(X|\theta)$, but can simulate from it. The algorithm works by sampling from the joint distribution of the parameter space by sampling from the prior and then sampling from the likelihood induced by the prior sample. Then the algorithm uses a machine learning conditional density estimator to produce the distribution of the parameters conditional on the the simulated data, $P(\theta|X)$. Conditioning the simulated data on the actual data produces the posterior.

Compared to the baseline Bayesian algorithm, Metropolis-Hastings Markov Chain Monte Carlo (MCMC), SNPE yields better posteriors with less iterations. It is widely known that MCMC is

1. For a review of heterogeneous agent solution methods that estimation approaches must be compatible with see, Reiter (2009), Winberry (2018) and Auclert et al. (2021*a*). Other heterogeneous agent estimation methods include Ahn et al. (2018), Liu and Plagborg-Møller (2023), and Kase, Melosi and Rottner (2022), but the problem is still difficult

difficult to use and takes a long time to converge (Herbst and Schorfheide, 2015), (Herbst and Schorfheide, 2014). MCMC randomly walks across the posterior with probability equivelent to the joint distribution, in propotion to the posterior. This means MCMC does ultimately sample from the posterior, but because of the random walk, over small time steps the samples exhibit serial correlation and the sampling procedure has difficulty traversing valleys to alternative modes because the vallies have low probability. At the same time, since SNPE doesn't sample from the posterior with a random walk, but instead learns the posterior via conditional density estimator, SNPE has none of these issues compared to MCMC. In fact, theoretical results suggests MCMC converges to multimodal and unidentified posteriors at an exponetial rate (Chewi, 2023), while it is likely SNPE is close to quadratic (Farrell, Liang and Misra, 2018), (Draxler, Schnörr and Köthe, 2022).

This matters because the derived posteriors matter for both policy and academic study. Here is an example of the marginal distribution of 4 of the 36 parameter Smets and Wouters (2007) DSGE model after five hundred thousand iterations of SNPE compared to MCMC after ten million iterations. The other 32 parameters are similar and in the results section. The y axis is probability mass. The x axis is the support of the posterior defined as ranging from the lower bound of the uniform prior for that particular marginal to the upper bound. The red line indicates the true value of the data generating parameter and the blue line is the posterior. The methodology for SNPE and MCMC are described in the III.B. and A section resepctively, and the Smets and Wouters (2007) are described in IV.F. section.

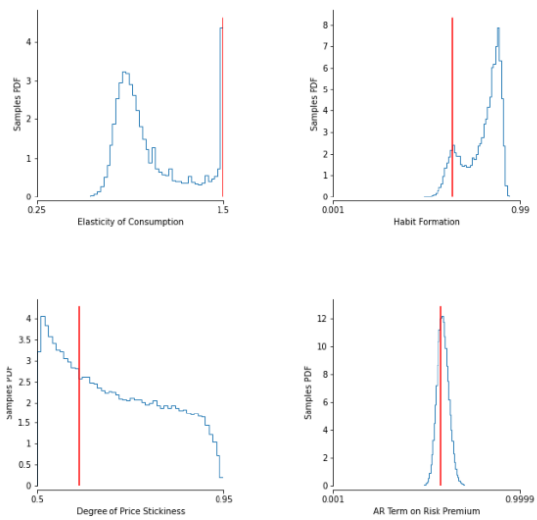

Figure I: Example of SNPE posteriors

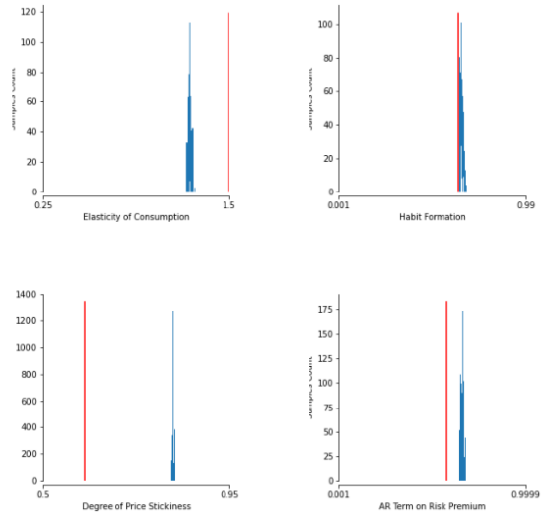

Figure II: Example of MCMC posteriors

Figure I: Comparison of SNPE Posteriors Compared to MCMC Posteriors

One argument that one could make is that the true posterior could be the tighter priors learned by MCMC. However, as one can see, often the MCMC posterior do not contain the actual data generating parameter, suggesting even after 10M iterations, the model is not close to have converged and the MCMC posteriros are much too tight. In contrast to MCMC, SNPE often matches the true parameter value with the mode–or at the very least–in the support. Some under-identification is expected as, in order to match real world macroeconomic data sets, only 200 timesteps were simulated and 36 parameters require estimation. Impressively, SNPE gets better results without access to the likelihood function that MCMC uses. When compared to non-likelihood simulation-based approximate Bayesian techniques (Rubin, 1984), SNPE converges to the posterior when the number of iterations goes to infinity Greenberg, Nonnenmacher and Macke (2019), when the approximate Bayesian approach does not.

It is a bit less natural, but one can use SNPE as an MLE estimator by using uniform priors and taking the mode of the posterior. When used as a simulation-based MLE point estimator, it also has some advantage over other simulation algorithms. The workhorse simulation algorithm is method of simulated moments (McFadden, 1989). This approach can estimate any method than can be

simulated, but is sometimes inefficient even when the model is correctly specified. The estimation method closest to SNPE is the GAN approach of Kaji, Manresa and Pouliot (2020). This method is often efficient with Kaji, Manresa and Pouliot (2020) providing the conditions, but it is slightly different than maximum likelihood. This means that the model likely will produce slightly different parameter values than MLE when misspecified. Kristensen and Shin (2012)also present a similar approach to my method, but they use a kernel estimator in their simulated maximum likelihood estimator. The kernel approach runs into curse of dimensionality issues as well as problems with conditioning. Unlike these simulation methods, SNPE also respects the state space generating structure of macro models, treating the data as a single data point time series. This is in line with how the the state space formulation of a solved structural model treats the time series data as a single observation with hundreds of lags. Compared to all the above approaches, my algorithm extends simulation-based approaches to work with Bayesian estimation and can MLE estimate even large models.

In summary, SNPE provides the advantages of full information with simulation-based estimation. Compared to Bayesian alternatives, SNPE produces better posteriors faster and can estimate without knowing the likelihood function. Compared to simulation-based approaches, SNPE is a full information approach that can estimate even complex models.

II. Literature Review

My paper fits in the intersection of simulation-based structural estimation, Bayesian inference, and the solution and estimation of HANK models. I will talk about simulation based estimation first. Then I will discuss Bayesian estimation. Finally, I will delve into the solution and estimation of heterogeneous agent models, including their relevance and current state of the literature. I will cover MCMC procedure and machine learning and their applications in the section following this literature review.

II.A. Simulation-Based Estimation in Economics

Simulation-based estimation of models, structural or otherwise, has been a prominent approach in economics for many years. Originating from method of simulated moments (MSM) (McFadden, 1989) simulation-based likelihood and inference techniques have been developed (Pakes and Pollard, 1989), (Duffie and Singleton, 1990) and improved to address the difficulty of model estimation. As a simulation technique, SNPE builds upon MSM. Maximum simulated likelihood (Lerman and Manski, 1981) (Hajivassiliou et al., 1997), (Haan and Uhlendorff, 2006) and other techniques have been used to improve efficiency, but still require the use of measurement error. Researchers have adopted these simulation-based methods to estimate dynamic models, particularly in industrial organization (Keane and Wolpin, 1994), as well as the ever-present usage for structural models in macroeconomics (Herbst and Schorfheide, 2015) and finance (Hennessy and Whited, 2007).

The issues of MSM have pushed the literature to search for ways to improve robustness to misspecification and efficiency. Even if a model is misspecified, one would like to obtain as narrow standard error bars as possible. Efforts have been made to develop efficient method of moment estimators using a spectrum of moments to guarantee that data is in the support of distributions (Gallant and Tauchen, 1996). An additional approach that this paper builds upon is Kaji, Manresa and Pouliot (2020), which allows for efficient estimation of structural models. A detailed comparison with how these models stack up with mine in the IV.A. table.

II.B. Bayesian Estimation

The baseline method for Bayesian estimation is MCMC (Herbst and Schorfheide, 2015). However, over the past ten years, there have been substantial improvements in Bayesian estimation. One of the first improvements involved the proposal by Herbst and Schorfheide (2014) to use the Sequential Monte Carlo approach–previously proposed for filter–to perform Bayesian estimation. This approach allowed for more effective estimation of multi-modal posteriors and faster convergence. However, this approach was complicated to set up and is still a likelihood method and requires among other things a filtering approach. This method both require a specific solution method and

have only been demonstrated to be effective with tight priors around the true parameter model. My appproach works well out of the box even with flat priors.

More recently, Childers et al. (2022) provide a way to differentiate through DSGE models solved via Schmitt-Grohé and Uribe (2004). For the time being, this approach doesn't work for heterogeneous agent models, however, the ability to differentiate through the DSGE model is a powerful tool. ignoring applications outside of Bayesian and MLE estimation, this approaches allows for the ability to perform MCMC sampling guided by gradient information like Hamiltonian Monte Carlo (HMC) and maximum likelihood directly without slower gradient free optimizers. Additionally, often more in other fields, the workhorse simulation-based Bayesian estimation approach is Approximate Bayesian Computation (ABC) (**?**). A detailed comparison of all these models is contained in the IV.A. table of the results section.

II.C. Heterogeneous Agent Models

The literature surrounding full information estimation of heterogeneous agent models is growing and an essential part of the future of macroeconomics. This section will focus on improving speed and accuracy of estimation methods. The demand for more accurate estimation approaches is significant, given the complexity of the models beings solved. For example, Ahn et al. (2018) proposes to use a PCA based approach to reduce the dimensionality of the latent space so one can filter and as discussed above, leads to losing information for estimation. Recently there have approaches that combine neural networks projection solution methods along with a complementary estimation approach (Kase, Melosi and Rottner, 2022), which I will discuss in the IV.G. section in the Reiter HANK method section of the results.

Another aspect of the heterogeneous models is the emphasis on computational speed of estimation. There are many papers proposing better solution methods (Auclert et al., 2021*a*), (Winberry, 2018), (Khan and Thomas, 2008), but their complexity means that it is more difficult to estimate with conventional methods. One innovations on this front is Liu and Plagborg-Møller (2023) which uses full information micro-data to complement macro variable estimation to perform true full-information Bayesian estimation. In particular, they estimate a Krusell and Smith (1998) model

via Winberry (2018) in a Bayesian manner. I build upon their approach and discuss advantages and disadvantages of our approaches in the IV.H. section of the results.

The issues highlighted by estimation–lack of accuracy, computational burden, and identification issues–are reasons so many heterogeneous agent modelers resort to calibration (Auclert et al., 2021*b*), (Ottonello and Winberry, 2020), (Caglio, Darst and Kalemli-Özcan, 2021). There is a need among both heterogeneous agent and representative agent model practitioners to reduce the barriers to maximum likelihood estimation and Bayesian estimation, and SNPE and related algorithms can help address these barriers.

III. Background on SNPE and MCMC

In recent years, Bayesian inference has gained increased prominence as a method for estimating models in economics. This section sheds light on two prominent techniques in this field, namely SNPE and the benchmark MCMC. To provide a comprehensive understanding, a brief overview of Bayesian estimation is first presented. The SNPE estimator is then discussed.

Despite its success in various applications, the last section will discuss the limitations and pathological cases for the SNPE estimator. While these methods have been widely used, it is essential to understand their limitations and make informed decisions when choosing the most appropriate approach for a particular problem.

III.A. Bayesian Basics

In this subsection, I will discuss the general paradigm of Bayesian estimation, then I will discuss MCMC.

Bayesian estimation attempts to find the posterior given the likelihood and prior of a model using Bayes' rule:

$$P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$$

Here $P(\theta|X)$ is the posterior, $P(X|\theta)$ is the likelihood, and $P(\theta)$ is the prior. All approaches

that perform Bayesian inference, ranging from MCMC, variational inference, to simulation-based inference concern techniques for calculating $P(\theta|X)$ without calculating the partition function, $P(X)$.

MCMC is a commonly employed technique for Bayesian inference in economics, as described in (Herbst and Schorfheide, 2015). The way MCMC gets around the intractable partition function is to evaluate the likelihood ratios between different parameter values, allowing for the determination of the relative likelihood of different points in the parameter space. As this is a likelihood ratio, one does not need to calculate the normalizing partition function, $P(X)$. This information is then utilized to direct a random walk, with points being visited in proportion to the relative likelihoods, effectively sampling from the posterior distribution $P(\theta|X)$. The benchmark method of Bayesian inference in economics is the use of this MCMC algorithm (Herbst and Schorfheide, 2015).

One difference between SNPE and MCMC is that the former assumes one cannot calculate the likelihood, $P(X|\theta)$, but only sample from it by sampling from the model. However the latter assumes that the likelihood can be calculated and MCMC is a approach that requires that additional assumption for estimation. Despite SNPE being disadvantaged by this assumption, even in cases where the likelihood is tractable, SNPE often produces posteriors faster that are more accurate when compared to MCMC.

III.B. Sequential Neural Posterior Estimation (SNPE)

This paragraph explains the details of the SNPE algorithm. First one draws a sample, θ, from the prior distribution, $P(\theta)$. Subsequently, simulated data, X, is generated from the model likelihood, $P(X|\theta)$. The concatenation of these two simulations results in samples from the joint distribution, $X, \theta \sim P(X, \theta) = P(X|\theta)P(\theta)$. Note this is equivalent to sampling from the joint distribution numerator in Bayes' rule. These samples can then be used to estimate the posterior distribution, $P(\theta|X)$, through the utilization of either a conditional density estimator like a mixture of Gaussians (Bishop, 1994), a normalizing flow density estimator (Mohammad-Djafari and Ayasso, 2009) or a GAN (Goodfellow et al., 2014). This section will both discuss the mixture of Gaussians and the normalizing flow as two examples of density estimators. Finally, conditioning $P(\theta|X)$ on

the real data, X', returns an estimate for the posterior: $P(\theta|X = X')$.

The use of more powerful machine learning conditional estimators, is responsible for much of the improvement over the previous literature. Kristensen and Shin (2012) sample from the joint distribution in a similar way as my method. However, they uses a kernel density estimator, which cannot estimate high dimensional distributions and doesn't allow for Bayesian inference. Likewise Kaji, Manresa and Pouliot (2020) are able to perform something similar to MLE estimation but cannot perform Bayesian inference and cannot adhere to the state space data generating assumption.

One can converge quicker if one uses multi-round inference to sample simulated data X in the neighborhood of the real data. Instead of sampling from the prior, θ is sampled from the best approximation of the posterior, leading to more relevant samples of X when X is sampled from the model conditioned on θ. However, this process requires importance sampling to correct for the shift in distribution as θ is no longer being drawn from the prior. The first round entails sampling from the prior, while later rounds involve sampling from the updated estimate of the posterior and applying importance sampling to account for the distribution shift. This can be mathematically expressed as:

$$P'(\theta|X) = P(\theta|X)\frac{P'(\theta)}{P(\theta)} \tag{1}$$

Here $P'(\theta)$ is the proposed distribution that one is sampling from, $P(\theta)$ is the true prior, and $P'(\theta|X)$ is the estimated posterior obtained by sampling from the proposal distribution. The use of a flexible machine learning density estimator, such as a normalizing flow or a conditional mixture of Gaussians, allows for the unbiased estimation of the posterior through the estimation of $f_\phi(\theta|X)$ in place of $P(\theta|X)$ in the above equation. When one samples for $P'(\theta)$ instead of the prior $P(\theta)$, the conditional distribution ones conditional density estimator is converging to is $P'(\theta|X)$ from ((1)). If one estimates a density estimator for $P(\theta|X)$, $f_\phi(\theta|X)$, on the distribution shifted samples $f_\phi(\theta|X)$ will converge to $P(\theta|X)$ after one first multiplies the samples by $\frac{P'(\theta)}{P(\theta)}$ first. See appendix A of Papamakarios and Murray (2016) for a proof and more information. Papamakarios and

Murray (2016) and Greenberg, Nonnenmacher and Macke (2019) have demonstrated that the use of universal approximators density estimators, such as normalizing flows (Huang et al., 2018) and conditional mixture of Gaussians (Bishop, 1994), in multi-round SNPE will converge towards the true Bayesian posterior in the computational limit under the KL divergence. Below is a description of the SNPE algorithm:

Algorithm 1: SNPE Algorithm

Input: Simulator $p(X|\theta)$, prior $p(\theta)$, data X', flow $f_\phi(X|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with Monte Carlo;

  Simulate $X^{(n)} \sim P(X|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{X^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(X, \theta)$ *not converged* **do**

    Sample $\{X^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Train $f_\phi(\theta|X)\dfrac{p^{(i-1)}}{p(\theta)}$ on $\{X^{(i)}, \theta^{(i)}\}_i^B$;

  end

  Update posterior $p^{(i)} \propto f_\phi(\theta|X = X')$;

end

There are theoretical reasons for the faster convergence of SNPE. It is widely known that MCMC or even improvements like HMC empirically converge slower than other machine learning alternatives (Nathoo et al., 2013). For multi-modal and under-identified distributions, random walk methods like MCMC, HMC, and No U-Turn Sampling (Hoffman, Gelman et al., 2014) do not converge in polynomial rate (Chewi, 2023). At the same time, the density estimator in SNPE uses maximum likelihood which suggests quadratic convergence, although SNPE may not converge to the global optimum. Nevertheless, results Farrell, Liang and Misra (2018) and Draxler, Schnörr and Köthe (2022) suggest similar polynomial convergence for algorithms like SNPE even in the case of multi-modal distributions.

III.C. Density Estimation

The SNPE approach proposed in this article utilize three distinct density estimators: Normalizing Flows, Conditional Mixture of Gaussians, and GANs. These methods are capable of handling high dimensional estimation in X and θ. The structural models are used to generate samples from X and θ, then with the exception of GANs, which uses the slightly different SNVI algorithm, each density estimator serves as the $f_\phi(\theta|X)$ function in the above algorithm.

One of the primary benefits of the proposed estimation algorithm is the improved handling of conditional and in some cases high dimensional estimation of θ. This allows for Bayesian and MLE estimation of models with up to 36 parameters, such as the Smets and Wouters (2007) model. This is a significant improvement over previous works, such as Kristensen and Shin (2012), which are limited in their ability to handle high dimensional and conditional density estimation.

Each of the three density estimator approaches has its own strengths and weaknesses. Mixture of Gaussians is the simplest model to implement and can handle most lower dimensional conditional density estimation with smooth and multi-modal distributions. Normalizing flows can handle exotic distributions with almost arbitrary dimensionality in θ and X. Using a GAN as a density estimator allows one to deal with locations in the support of the posterior where the model doesn't converge. Since there is no data in this space, the other techniques will extrapolate values and perhaps even put positive support in these areas. The GAN approach pared with projecting the derived distribution onto a normalizing flow using the reverse Kullback-Leibler divergence, dubbed Sequential Neural Variational Inference (SNVI), results in fast inference combined with the ability to assign no probability to places where sampling fails. The issue with this projection is that minimizing the reverse Kullback-Leibler divergence is known to lead to distributions that concentrate too much mass at the modes and too little at the tails. This is the main weakness of SNVI. SNVI is further discussed in the CD. section of the appendix.

III.D. Conditional Mixture of Gaussians

The conditional mixture of Gaussians approach is a powerful tool for conditional density estimation and, of the three methods, is the easiest to explain. As outlined in Bishop (1994), a mixture of Gaussians is a linear combination of N Gaussian distributions, each with its own mean, covariance, and weight. I will first start by discussing unconditional mixture of Gaussian density estimations. Here is a picture of how graphically mixture of Gaussians fits a distribution:

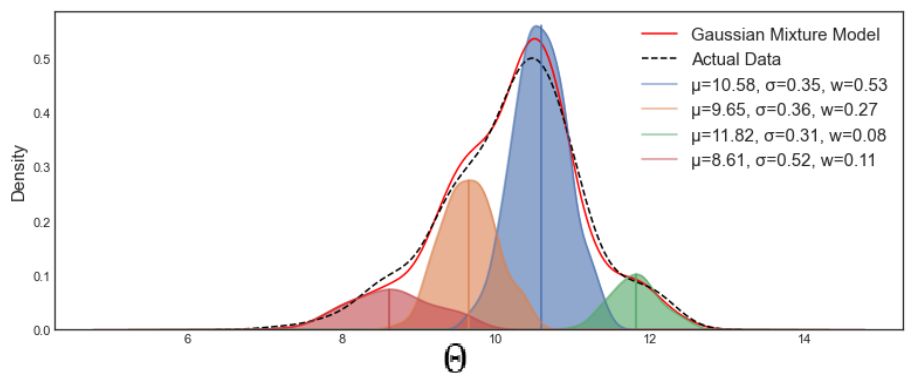

Figure II: Illustration of how a Gaussian Mixture Model fits a Distribution

The y axis is probability mass and the x axis is parameter value. This is an example of 1 dimensional density estimation for intuition, but the mixture of Gaussians can handle up to around 10 dimensional density estimation. As one can see, the Gaussian mixture is a weighted average of Gaussians with differing means and standard deviations. By modifying the weights, means and covariance matrices, one can combine a number of Gaussians to accurately represent a distribution. The probability density function (pdf) of a point under this mixture can be calculated using the equation:

$$P(\theta) = \sum_i \pi_i * Q_i(\theta)$$

Where $Q_i(\theta)$ is a normal distribution with mean and variance (covariance) specific to each Gaussian component. This represents the unconditional density estimator, however, the algorithm requires a conditional density estimator. This can be achieved by using an optimizable function,

such as a neural network, that takes the X data as input and returns the parameters for the mixture of Gaussians. If one is unfamiliar with feed forward neural networks, there is information in the AA. section of the appendix, however, for intuition, a high dimensional polynomial regression or spline would also be able to learn this conditional distribution. The function would then return the weights $\pi_i(X)$, means $\mu(X)$, and covariances $\sigma(X)$ for each Gaussian component, resulting in the following conditional density estimator:

$$P(\theta|X) = \sum_i \pi_i(X) * Q_i(\theta|\mu(X), \sigma(X))$$

Finally, I will discuses the estimation procedure. The parameters of the neural network can be optimized via maximum likelihood estimation (MLE), maximizing $P(\theta|X)$ and improving the accuracy of the density estimator. The approach maximizes $P(\theta|X)$ given the simulated conditioning data X. The feedforward neural network will learn a mapping from X to the parameters of the mixture of Gaussians (ie $\sigma(X)$, $\mu(X)$ etc.). The parameters of the feedforward neural network is then optimized so that the mixture of Guassians produces the highest likelihood for the θ variables.

The problem of this approach is that it runs into curse of dimensionality problems quite quickly. For example, Smets and Wouters (2007) has 36 parameters. With a mixture of Gaussians each covariance matrix of a Gaussian would have dimensionality $37 * 36/2 = 666$. Thus for higher dimensionality problems, one will either have to use flows, as I generally do, and explain next, or a GAN-discussed in the CB. section of the appendix.

III.E. Normalizing Flows

I will frame this section discussing a particular type of normalizing flow, the neural autoregressive flow (Huang et al., 2018). Then I will discuss how to use the change of variables formula to learn a mapping from realizations of random variable to a samples from a target distribution one wants to estimate. Using the change of variables, one can derive a likelihood of a sample under the flow, allowing one to ultimately perform likelihood estimation to perform density estimation. I will also attempt to highlight advantages flows have over KDEs that make the SNPE algorithm

more powerful that the traditional Kristensen and Shin (2012) approach, including the ability to do conditional density estimation, high dimensional density estimation, and the ability to sample and maximum likelihood estimate without resorting to rejection sampling.

The main idea behind a normalizing flow is to compose a series of k transformations, f_k, one on top of another to map realizations from a known distribution, z_0 which could be sampled from a distribution like $N(\bar{0}, \bar{1})$, to realizations of a distribution of interest. I will call this distrubution z_K as there are K transformation and the output of the last transformations is z_K. The z_K pdf is ultimately what I am trying to estimate and when I've explained the model in detail will the discuss equating the z_K variable with θ as the realizations of the parameter distribution I am using the normalizing flow to density estimate. As I will discuss later, this is done by optimizing the parameters of f_k to maximize the likelihood that samples from the unknown distribution $\theta = z'_K \sim p(z'_K)$, which should be matched after estimation by the z_K output from the series of K transformations: $z_K = f_{K-1} \circ f_{K-2} \circ ...f_0(z_0)$.

Take a function f_k which takes in realizations,$z_i = k$, from a random variable and produces samples z_{k+1}. The formula for $P(z_{k+1} = f_k(z_k))$ given $Q(z_k)$ is the familiar change of variables:

$$P(z_{k+1} = f_k(z_k)) = Q(z_k = f_i^{-1}(z_{k+1})|det\frac{df_k^{-1}(z_{k+1})}{dz_{k+1}})| = Q(z_k)|det\frac{df_k(z_k)}{dz_k}|^{-1} \tag{2}$$

Given a series of transformations $z_K = f_{K-1} \circ f_{K-2} \circ ...f_0(z_0)$, then the formula for the log likelihood of the whole sequence of change of variables is:

$$ln(q(z_K)) = ln(p(z_0)) - \sum_{k=0}^{k=K-1} ln(|det\frac{df_k(z_k)}{dz_k}|) \tag{3}$$

Notice how we can sample $z'_K \sim P(Z'_K)$ then equate z_K with z'_K and evaluate the likelihood of z'_K under the flow. In this way, after optimizing the flow, the distribution of z_K points will match the distribution of the points z'_K. This process though, requires the inverse process as the likelihood of a point z'_K is dependent on calculating z_0. Thus one has to be able to invert the flow

in addition to being able to sample from the flow via the forward process. This means that each f_k must be a invertible function (ie a bijection). Thus, each f_k is called a bijector and a chain of f's in conjunction with the realizations of the initial known base distributions, z_0, is called a flow. Given this information, we can create a flexible function that can convert realizations from a simple distribution–for example a multivariate normal with mean vector $\bar{\mathbf{0}}$ and standard deviation vector $\bar{\mathbf{1}}$–into realizations of many different "non-parametric" distributions. Below is a picture of how the flow can transform a 2 dimensional normal random variable in a distribution whose scatter-plot spells the word BRAD:

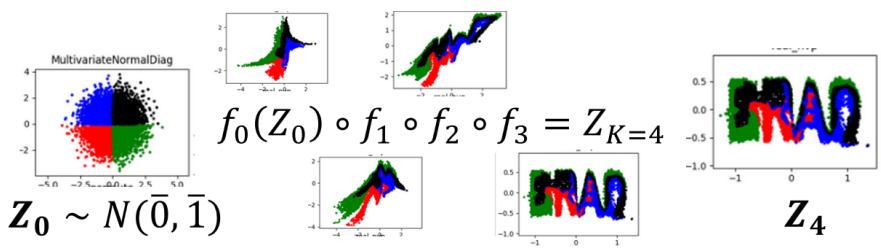

Figure III: This Flow is Transforming Realizations From a 2-D Normal, into a Distribution Whose Scatter-plot Spells BRAD

The x and y axis of all the pictures in the BRAD flow example illustrate the realizations of the sample points along both axises of the flow. z_0 is a 2-dimensional Gaussian and thus because the flow is invertible, each intermediate z_k up to the output z_K are two dimensional. Likewise the picture above and below each f_k illustrate the slow transformation of the normal noise into the BRAD scatter-plot at the end. This is a much more difficult distribution to estimate than SNPE posteriors which have continuous, full, and connected support, and if they are multi-modal, in a much more straightforward way than this BRAD distribution. However, theoretical results support the flexibility of normalizing flows. Huang et al. (2018)–the model this paper uses–shows that by chaining together f_k's as a flow, under some assumptions, can produce realizations from any continuous distribution by transforming it from normal distribution realizations.

The next section deals with invertibility of f_k. f_k is a J dimensional vector valued function that takes in the J dimensional z_k vector and returns a J dimensional z_{k+1} vector. Some syntax: A

subscript under a variable refers to the layer in the flow, ie the k that corresponds to which f_k as there are capital K f's composed one on to of the other. The input to each f_k is a z_k. As both z_k and f_k are vectors/vector valued functions, the supercript, j, indicates the element j of the vector z_k, with, as implied earlier, J being the size of the vector z_k.

A simple but inflexible function that is a bijection is an affine transformation:

$$z_{k+1} = a_k * z_k + b_k \tag{4}$$

and its inverse:

$$z_k = \frac{z_{k+1} - b_k}{a_k} \tag{5}$$

This cannot model conditional distributions and more nuanced distributions compared to linear transformations of Gaussians (which are still Gaussians) if the base distribution is a multivariate Gaussian. Here is an example of a three element f_k function that has constitutionality and more nuance:

$$z^1_{k+1} = a^1_k * z^1_k + b^1_k // z^2_{k+1} = a^2_k(z^1_k) * z^2_k + b^2_k(z^1_k) z^3_{k+1} = a^3_k(z^1_k, z^2_k) * z^3_k + b^3_k(z^1_k, z^2_k)... \tag{6}$$

The important thing to note is the function on the first element of z_k is still affine. However the a^j_k and b^j_k terms, with j greater than 1, are functions, that take in z terms and output coefficients. The function on the second element a^2_k and b^2_k are dependent on z_1. Likewise equation 3 is dependent on z_1 and z_2. Additional equation follow this same pattern. These a_k and b_k functions are feedforward neural networks, which are discussed in the appendix. However for intuition, these function could be high dimensional polynomial regressions and the parameters to optimize are the linear βs. This structure allows the flow to remain invertible. To see this first assume one knows all the capital J elements of z_k, since the right hand side is entirely a function of all J elements of the z_k vector,

one can calculate all the terms in z_{k+1}. The reverse is more complicated. Now one knows all the elements of z_{k+1} and needs to find the values of z_k. Since z^1_{k+1} is known, one can invert the first affine equation ($z^1_{k+1} = a^1_k * z^1_k + b^1_k$) to get the value of z^1_k. Now that z^1_k is known, a^2_k and b^2_k only take in z^1_k, so one can invert the j = 2 equation: $z^2_{k+1} = a^2_k(z^1_k) * z^2_k + b^2_k(z^1_k)$ to get z^2_k from z^2_{k+1} and z^1_k. Now that z^1_k and z^2_k are known one can recover z^3_k from the third equation. The proof that further elements are recoverable proceeds by induction. One can show, with slight additional modifications, that the flow described here, a rough approximation of the neural auto-regressive flow, allow SNPE and related algorithms to converge under the KL divergence to the Bayesian posterior (Greenberg, Nonnenmacher and Macke, 2019). Similar results are true for GANs and mixture of Gaussians (Papamakarios and Murray, 2016).

III.F. SNPE Theory

Proposition 1: Mixture of Gaussians are a universal approximator of continuous distributions. See Barron (1993) for the proof. See urlhttps://mjt.cs.illinois.edu/dlt/#classical-approximations-and-universal-approximation for an explanation of how the proof extends to a mixture of Gaussians.
Proposition 2: A neural auto-regressive normalizing flow with some regularity conditions (ie for example having sigmoid and inverse sigmoid non-linearity) is a universal approximator of continuous distribution
See Huang et al. (2018) Appendix F and G
Proposition 3: SNPE algorithm will converge to the true posterior if the posterior is contained in the parameterization of the posterior conditional density estimator.
See Papamakarios and Murray (2016) proof of proposition 1 and note it is an application of the consistency of maximum likelihood estimator applied to the posterior conditional density estimator.

III.G. Caveats and Pathological Examples Using SNPE

There are certain limitations associated with SNPE that must be considered when applying it to a specific problem. One of the limitations of SNPE is its inability to handle the discontinuities that

are present in some models with kinks, such as s-S models. This leads to a lack of smoothness in the posterior of the kinked models, and the assumption that there is a normalizing flow parameterization that can generate the posterior is violated. In practice, a flow still converges, but it learns a continuous function that approximates the discontinuity in the posterior. This approximation can still be near the true likelihood, but if estimation with a discontinuous model is a problem, one can also add measurement error to smooth the problem out.

Another limitation of SNPE is its difficulty in handling stochastic singularities, which also result in estimation issues. When this occurs, the dynamic model learns a lower dimensional sub-manifold in the entire space, by assumption, the normalizing flow or mixture of Gaussians has to have positive support on the entire space of the target distribution. As in the case of the BRAD scatter-plot, the biggest problem isn't having regions of zero support, but having a support that is a lower dimensional manifold of the proposed space. In this case the flow will have to learn a "knife's edge" distribution which will be difficult. In cases with these stochastic singularities in a model estimated on real data, the model will fail to match the data. However, one can add more shocks, add measurement error, or make a change in modeling to avoid a stochastic singularity (Caballero and Engel, 2007) and allow full support over the range of the model output.

IV. Results

This section will evaluate the performance of SNPE by examining the proper behavior of the posterior and inspecting its accuracy. To this end, seven models are estimated using the SNPE algorithm, each showcasing a unique aspect and characteristic of the algorithm. The first model is the RBC model that I will walk through my approach with. The second model is a toy corporate finance model that is solved via value function iteration. The third model is the Lucas asset pricing model solved through projection. The fourth model is a bequests model solved via value function iteration. These four models are discussed mostly in the appendix. The fifth model is the Smets-Wouters model, which is estimated through both the MCMC and SNPE algorithms. The sixth model is the HANK model solved through Reiter's method and time iteration. The seventh model

improves on full information estimation of a Winberry (2018) model speeding up the approach taken by Liu and Plagborg-Møller (2023). This model is extended to an empirical application estimating both micro and macro moments in a full information setting. The details of the construction and implementation of these models can be found in the appendix.

In the empirical analysis, the accuracy of the posterior is inspected by comparing it with the calibrated parameters producing the simulated data. In theory, through the Bernstein von Mises theorem, the posterior should converge to the true values, although the structural models may not satisfy the assumptions of the theorem. Although the correct posterior should converge to the true parameter value as the number of data samples increases, finite sample sizes will not result in a perfect match. Hence, it is important to assess the posterior mode and ensure that it generally matches the true value and that there is no vanishing posterior mass at the true values. This check provides useful information to determine the performance of SNPE in practice and how it compares with other estimation techniques, such as the MH-MCMC algorithm. While in many cases the method is tested using simulated data, in order to resemble real world data, 200 time steps are sampled from the simulated model, to maintain a rough correspondence to the length of real world macroeconomic data. This small number of time steps implies that some parameters will be inadequately identified especially in the models with a large number of parameters. For the sake of illustrating the estimation benefits, all priors will be uniform over the range specified in the graph, making the estimation problem difficult to demonstrate modeling capability. First I will provide some charts demonstrating the capabilities of SNPE compared to simulation based algorithms and then Bayesian algorithms.

IV.A. Adaptability of SNPE to Estimation of Models Solved via a Variety of Methods

In order to depict the advantages of SNPE, I provide charts depicting the capabilities of different models. As described in the intro, SNPE can perform simulated maximum likelihood estimation (SMLE) by taking the mode of the posterior with flat priors. Additionally it can perform Bayesian estimation. Thus I have two tables: one comparing SNPE to other simulation based methods and

another comparing SNPE to Bayesian methods. These charts will hopefully illustrate, whether you want to do frequentist simulation based estimation or Bayesian estimation, SNPE will be a strong choice for the algorithm to use.

Here is the legend:

Symbol	Meaning
✓	Model can do this without difficulties
$\sim$	Model comes close to or can with modifications
?	Could not find research
✗	Model cannot do this

First, comparisons with simulation-based approaches:

Solution Method/Algorithm	**SNPE**	**MSM**	**K & S**	**GAN**
Estimate small scale DSGE	✓	✓	✓	✓
Estimate medium/large DSGE	✓	✓	✗	✓
Estimate HANK - Reiter	✓	✓	✗	✓
Estimate HA - Winberry	✓	✓	✗	✓
Perform SMLE	✓	✗	✓	$\sim$
Adhere State Space DGP	✓	✗	✗	✗

Here K & S indicates Kristensen and Shin (2012) and GAN indicates Kaji, Manresa and Pouliot (2020). the first two criteon that all th models are evaluated against, is the ability to estimate a small and medium/large DSGE model. The third and forth criterion is the ability to estimate HA models in two different ways. The fifth method is the ability to perform simulation based MLE and the last criterion is the ability to adhere to the state space data generating process for structural models. Other than ABC shown below, these approaches are simulation based but do not have Bayesian extensions and, mentioned earlier, a serious problem with these models is that they do not respect the state space assumption of structural models (criterion six). Instead in order for

these approaches to properly estimate, they add a cross sectional dimension, where each time step in question has a small number of lagged data and each data point is a separate time step with these lags. This would be the proper way to estimate a VAR model, but because solved structural models have a state space structure, they should be estimated with the entire time series as a single data point. Unsurprisingly, SNPE seems to be more flexible and powerful compared to competing methods.

Second a comparison of Bayesian approaches:

Solution Method/Algorithm	**SNPE**	**MCMC**	**HMC**	**SMC-est**	**ABC**
Estimate small scale DSGE	✓	✓	✓	✓	∼
Estimate medium/large DSGE	✓	✓	✓	✓	∼
Estimate HANK - Reiter	✓	∼	✗	∼	∼
Estimate HANK - Winberry	✓	✓	✗	?	∼
Estimate without likelihood function	✓	∼	✓	∼	✓
Quality posteriors even with large models	✓	✗	∼	✓	∼

In addition to SNPE and the benchmark MCMC, I compare with the HMC approach of Childers et al. (2022), the SMC estimation of Herbst and Schorfheide (2014), and approximate Bayesian computation Rubin (1984). The first for critereon are the same as for the previsou table. The fift criterion is the abiliyt to do Bayesian estimation from simulation only and the final critereon is to have quality posteriors even with large models. As one can see, the SNPE outperforms the other methods on the ability to perform all these critereon.

IV.B. RBC Model

I will walk through the RBC model using my algorithm. First, as with all of my models, I use flat posteriors defined over an appropriate range. In this case the parameters: α - captial share, β - discount rate, δ - depreciation rate, and ρ - productivity persistence parameter, all range from 0 to 1. After sampling parameters from this 4 way uniform distribution, I sample simulated data

from my RBC model, conditioned on the parameters I sampled from the prior. The RBC model produces simulated data: productivity, interest rate, capital, output and consumption. The joint distribution of the sampled parameters and the simulated data is used to train a conditional density estimator that learns the pdf $P(\theta|X)$, where θ is the simulated parameters, and X is the simulated data. Then conditioning X on the real data X', one learns the posterior: $P(\theta|X = X')$. This is essentially the SNPE process. Here is the chart of the posterior distribution - more graphs and information is provided in the appendix:

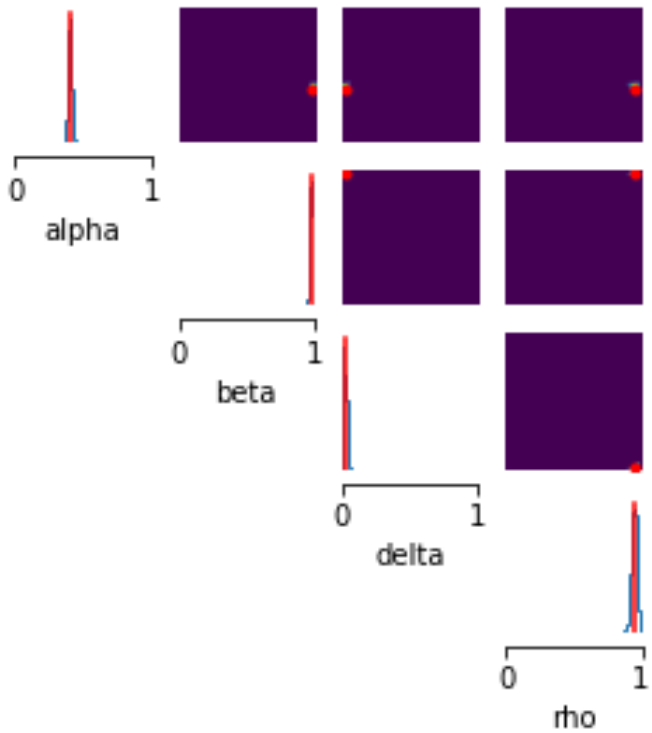

Figure IV: Simulated Data RBC

The diagonal graphs indicate the marginal distributions of the posteriors. The y axis on the graphs illustrates increasing probablity mass and the x axis indicates the support of the posterior which ranges from the lower bound to the upper bound of the uniform prior. The off-diagonal charts are two way contour plots where the x axis corresponds the the marginal distribution at the

same vertical axis and the y axis corresponds to the marginal at the same horizontal axis. The green and yellow colors on this plot indicate increasing probability mass and the blue indicates little or no probability mass.

IV.C. Cash Flow Model

Here is the posterior picture that comes from estimation of the cash flow model:

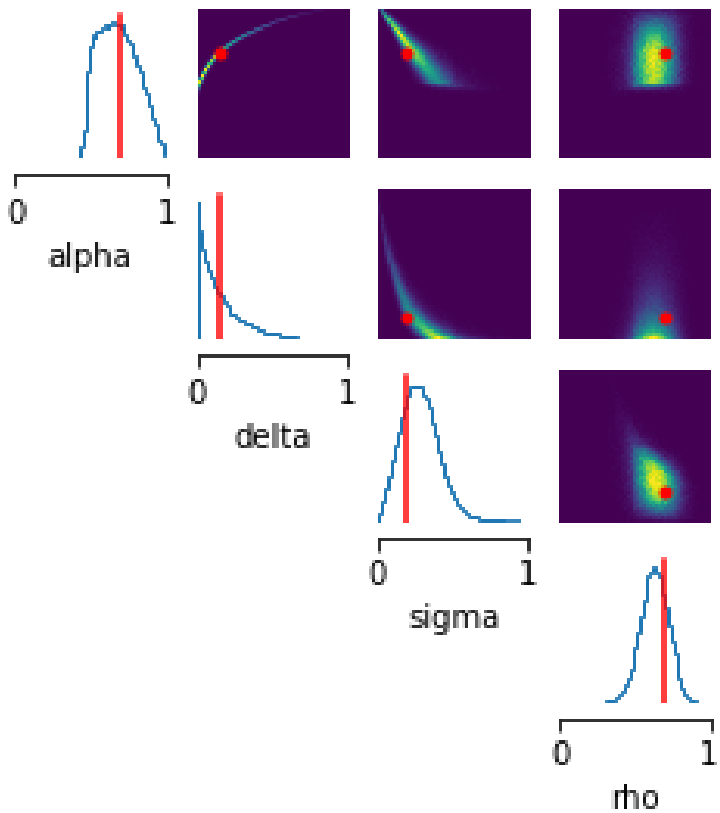

Figure V: Value Function Iteration with a Normalizing Flow Density Estimator

More details are found in the appendix.

IV.D. Projection - Lucas Tree

Here is the posterior picture that comes from the cash flow model

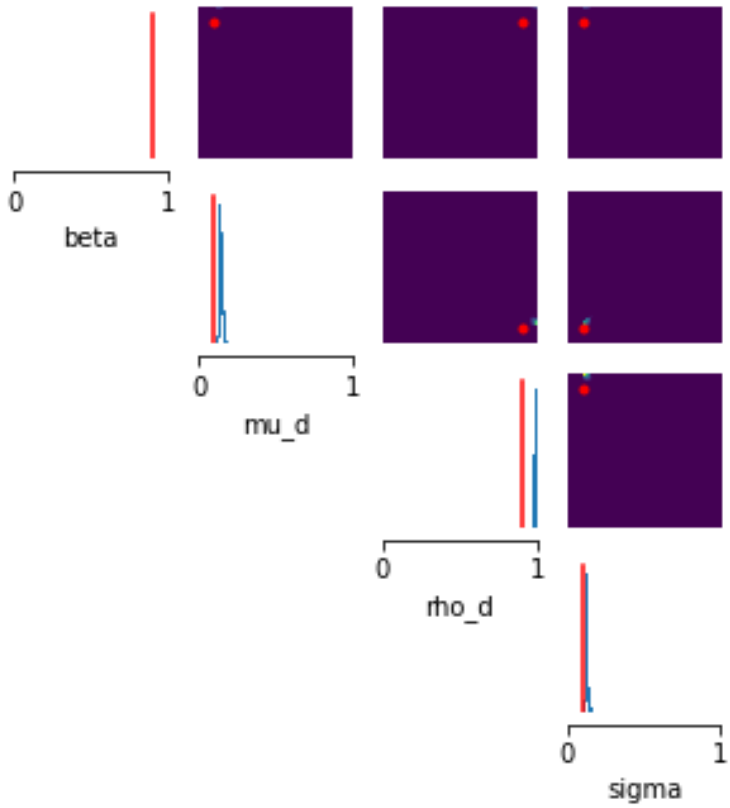

Figure VI: Projection Model

More details are found in the appendix.

IV.E. Value Function Iteration - Bequests Model

The seventh model considered in this study is an 11-parameter bequests model. This chart corresponds to the parameters in the model appendix section of the paper.

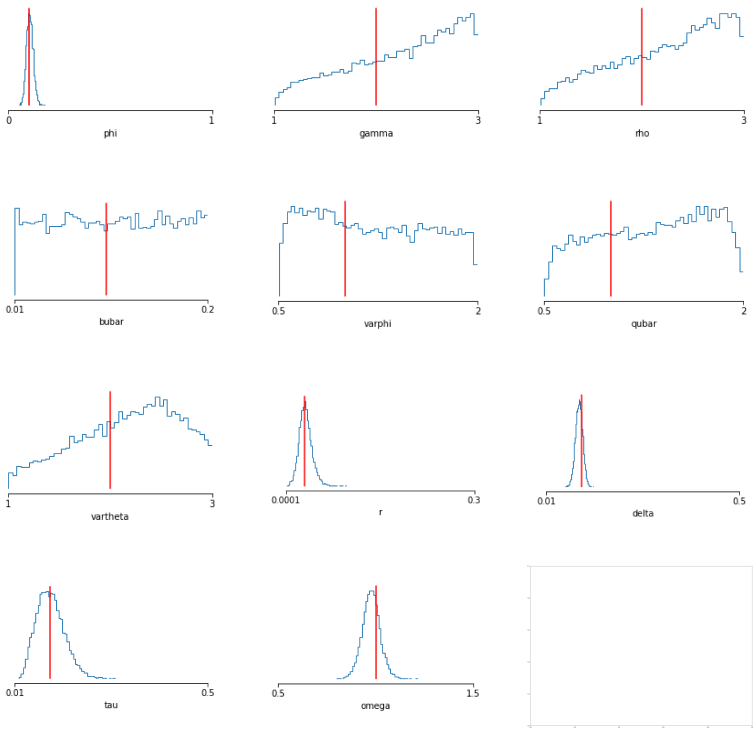

Figure VII: Eleven Parameter Model Estimated via VFI

The results of the estimation of the model parameters via value function iteration (VFI) are shown in the figure. Although many of the parameters have converged to the true values, some of the parameters still exhibit high uncertainty. This uncertainty is due to the fact that the parameters have very flat likelihoods. Many of the posterior distributions do concentrate around the true values, suggesting that the model has some discerning power with regard to the parameters. Next I will discuss the main results estimating the Smets-Wouters 2007 model as well as two different HANK models with one HANK empirical application.

IV.F. Smets-Wouters 2007 Model

The Smets-Wouters (2007) DSGE model is a representative agent New Keynesian model that describes an economy with a flexible price sector and a sector with sticky Calvo prices. The model has eight observed variables and eight shocks that interact with the rest of the model in an auto-

regressive manner. The linearized form of the model has 36 parameters, which were estimated using roughly 500,000 samples generated by the posterior. The parameters are separated into three different charts of 12 parameters each, and only the marginals are displayed to avoid visual clutter. These graphs are histograms rather than smoothed with kernel. This illustrates that the density estimators, be they Gaussians or flows, generate natural looking posteriors, which is not something MCMC generates. The variable labels are difficult to read, due to the effort to provide a reasonable number of parameters in each page. The BE. section of the appendix has a list of variables in the order they are listed.

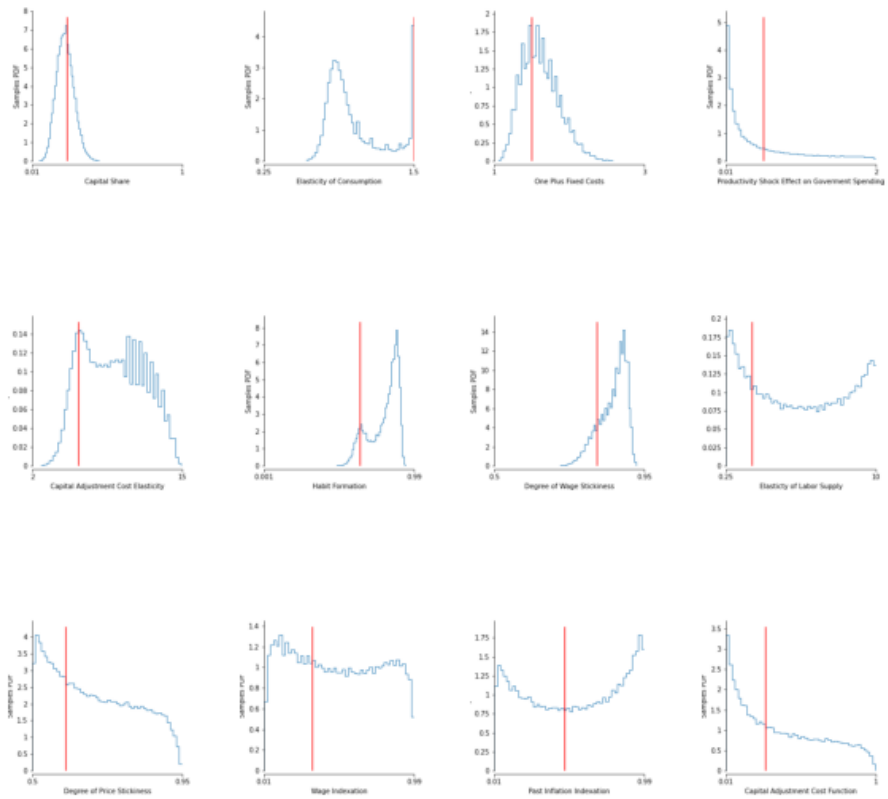

Figure VIII: Smets-Wouters Posterior with Parameters 1-12

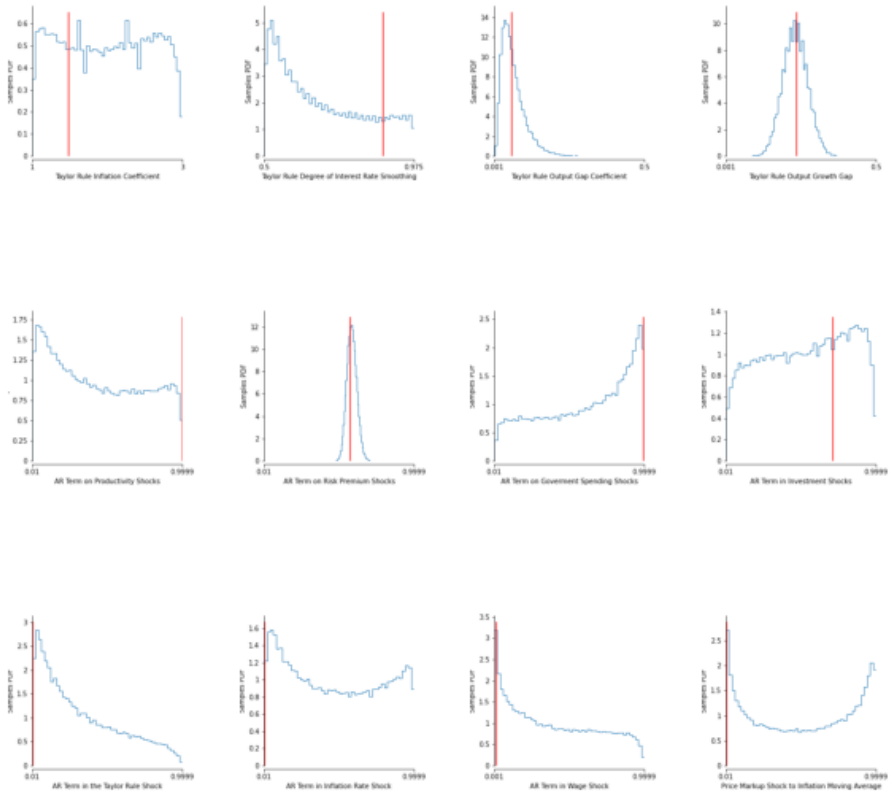

Figure IX: Smets-Wouters Posterior with Parameters 13-24

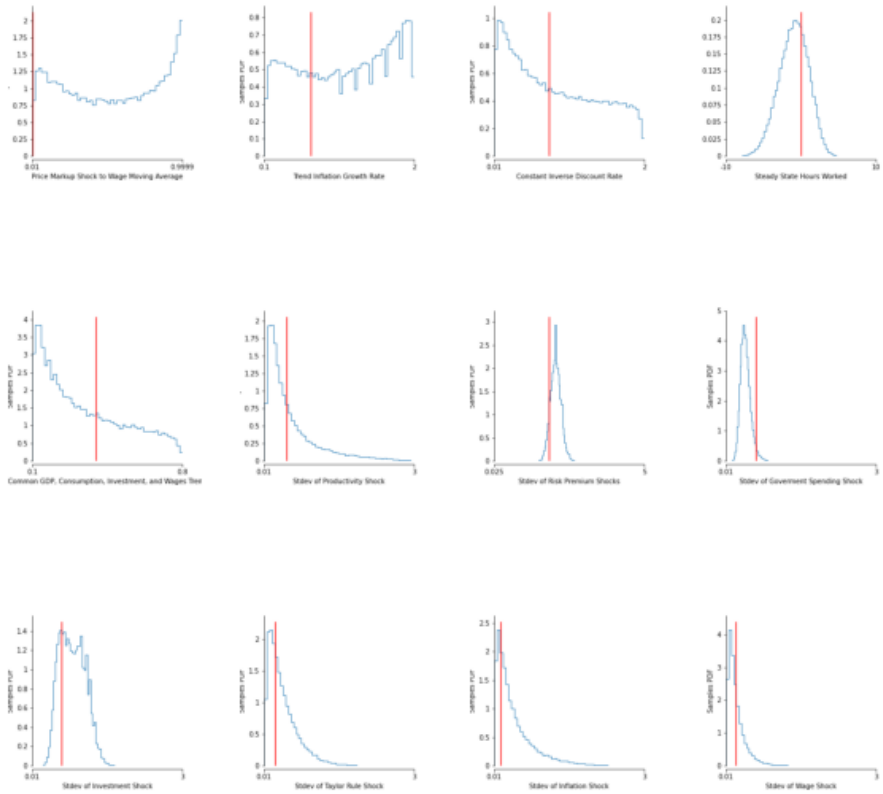

Figure X: Smets-Wouters Posterior with Parameters 25-36

The Smets-Wouters model demonstrates varying levels of success in estimating the various parameters. For example, the model accurately estimates the capital share but has more difficulty with the adjustments costs elasticity. The (one plus) fixed costs are estimated well. The Taylor rule parameters for output gap and change in output gap are estimated well too, while the inflation Taylor rule parameter is diffuse. Some of the autoregressive parameters for the shocks are well estimated, while others are diffuse or located in areas of low probability mass. The model performs well in estimating the standard deviation of the shock variables.

The posteriors estimated by the Smets-Wouters model with MCMC are less successful in covering the true data generating process parameters. Despite the use of state-of-the-art Dynare software (Adjemian et al., 2022) to draw samples, the model took roughly 30 hours to draw 10 million

samples, and the posterior showed limited coverage even after ten thousand sample burn in per chain. Below I reproduce the (Smets and Wouters, 2007) using MCMC instead.

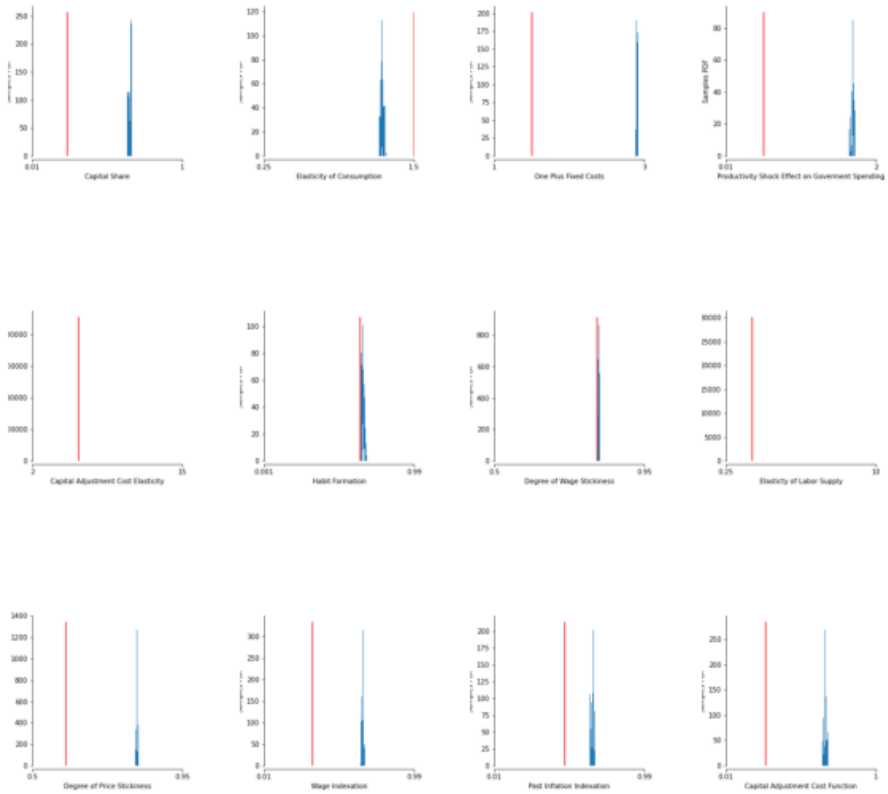

Figure XI: Smets-Wouters MH-MCMC Posterior with Parameters 1-12

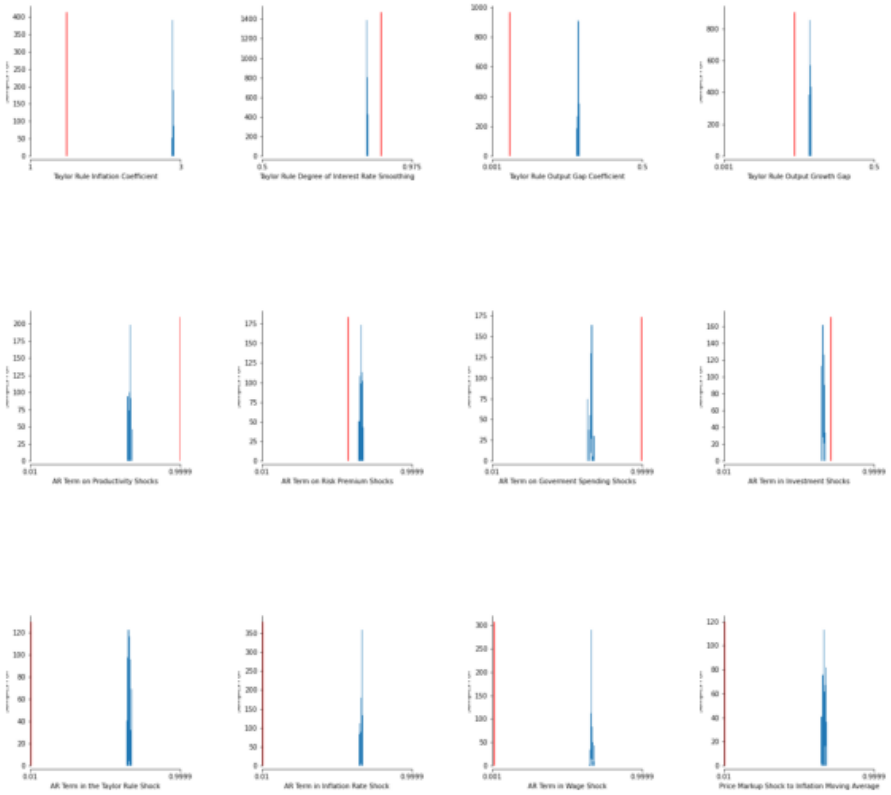

Figure XII: Smets-Wouters MH-MCMC Posterior with Parameters 13-24

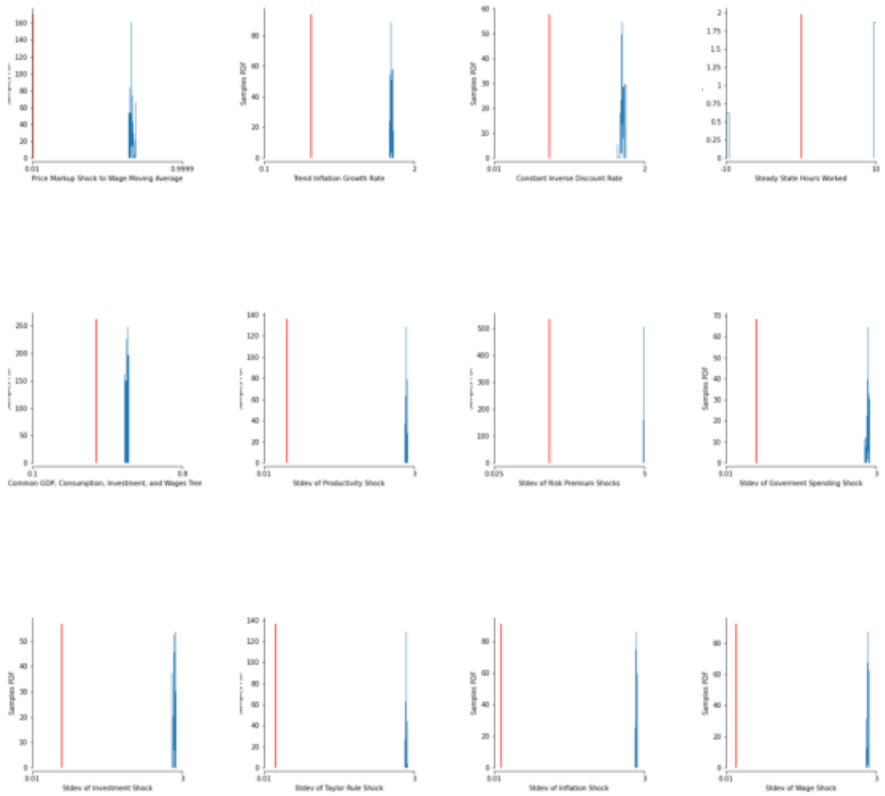

Figure XIII: Smets-Wouters MH-MCMC Posterior with Parameters 25-36

The performance of the MCMC algorithm, as applied in this study, presents several challenges in terms of its ability to converge to the true data generating parameters. While some of the posteriors show good coverage of the true parameters, a substantial number of the posteriors exhibit a support outside of the ground truth and often converge to delta functions far from the true parameters. In particular, the AR term in wage shock equation and price markup shock variables stand out as presenting the most glaring errors, with the MCMC algorithm placing all its mass near .8, whereas the true parameter is located at .015. On the other hand, the SNPE posterior presents a more reasonable representation, putting some mass at .99 and also exhibiting a mode at .015 for both variables. This mistake is repeated to a lesser extent for other variables such as Taylor coefficient on output gap growth, Taylor coefficient for inflation, constant trend inflation growth rate, steady

state hours worked, standard deviation of productivity, among many others. The posteriors in SNPE have no issue reproducing multi-modal posteriors, a documented problem with MCMC and is illustrated in the images of Herbst and Schorfheide (2014).

In the cases of informational priors, there is much evidence for the lack of convergence under realistic computational resources for MCMC. For example, Herbst and Schorfheide (2014) demonstrate the identification problem for MCMC in the Smets-Wouters model, while Andrews and Mikusheva (2015) highlight theoretical problems in identifying DSGE models in the MLE setting. When using MCMC-based simulated annealing to perform MLE optimization, Siarry et al. (1997) demonstrate curse of dimensionalty issues even with objective functions with as few as 3-6 parameters. Additionally **?** points out that for multi-modal or even unidentified distributions, MCMC converges at a rate slower than polynomial, while there is reason to suspect SNPE converges quadraticly or perhaps slightly slower but still at a polynomial rate (Farrell, Liang and Misra, 2018), (Draxler, Schnörr and Köthe, 2022).

The SNPE algorithm used in this study was implemented in Python, while the MCMC algorithm was performed in Dynare with pre-loaded Smets-Wouters code from the authors, with only the priors changed to uniform priors and some estimation settings modified. However, due to the difference in programming languages and the overhead of calling Matlab code in Python, the estimation process was much slower for the SNPE algorithm, with roughly 2 models solved per second, compared to about 100 models solved per second for the MCMC. If the solution method in both algorithms were written in Python, MCMC would be significantly slower than it is now. Nevertheless, to avoid any suggestion of unfairness comparison, the SNPE posterior is shown below after 150 thousand samples, which took roughly the same 30 hours as the 10 million samples of the MCMC:

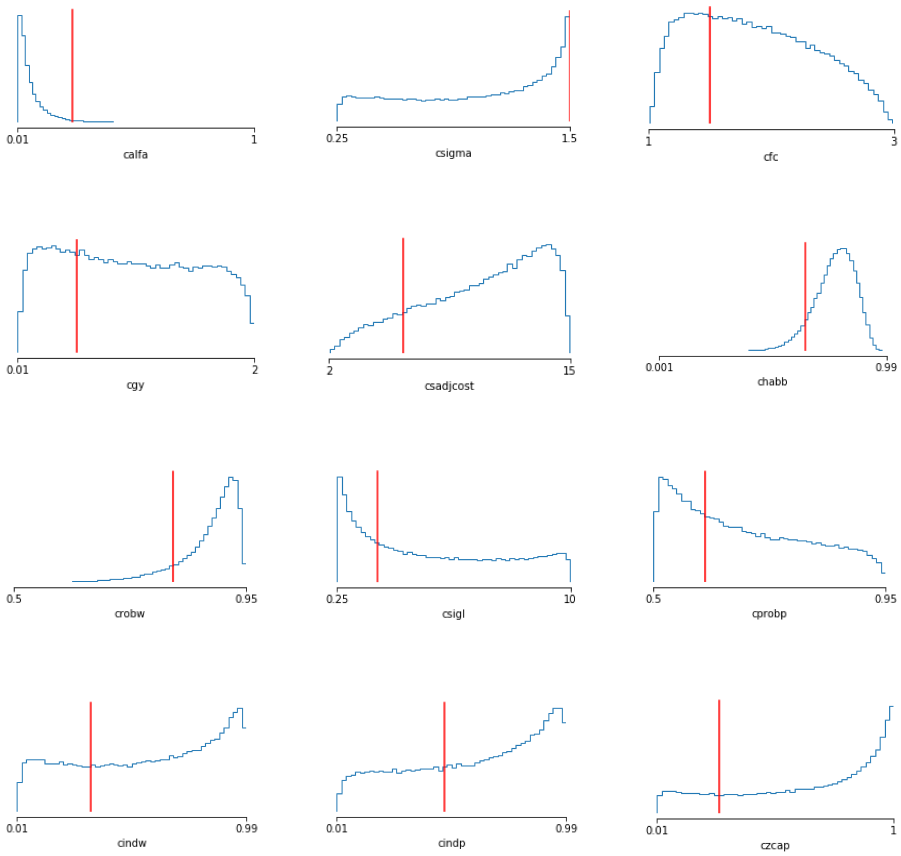

Figure XIV: Smets-Wouters SBI Posterior with 150 Thousand Samples: Parameters 1-12

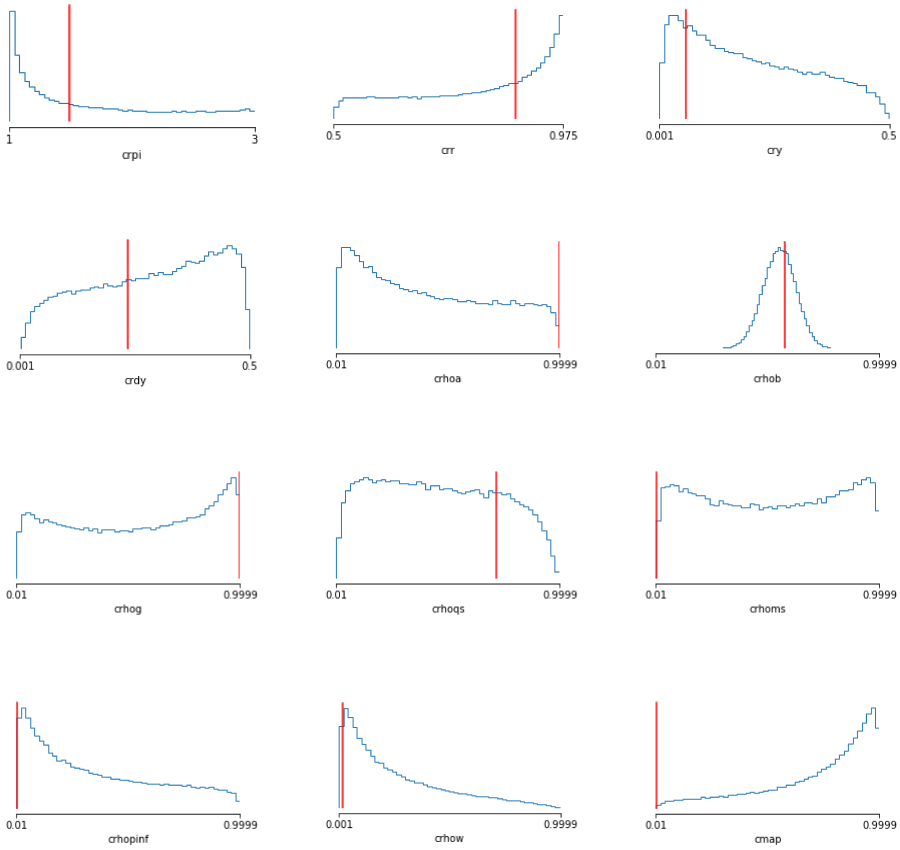

Figure XV: Smets-Wouters SBI Posterior with 150 Thousand Samples: Parameters 13-24

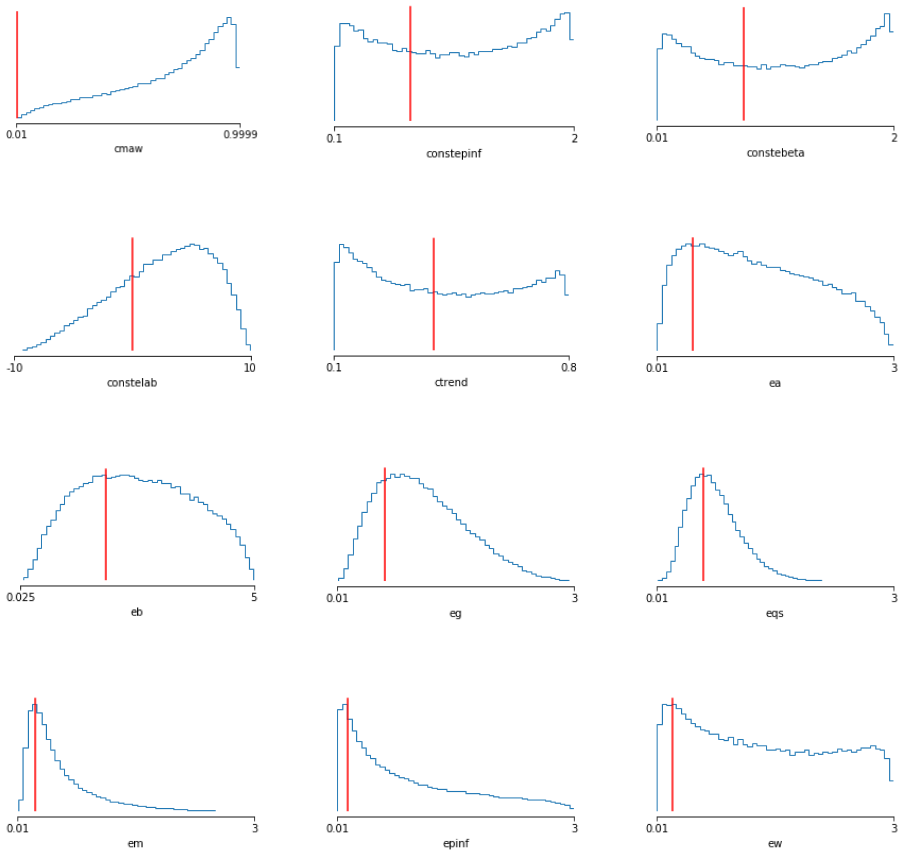

Figure XVI: Smets-Wouters SBI Posterior with 150 Thousand Samples: 25-36

Although the SNPE algorithm incurs a time penalty, because it is written in Python, it still manages to learn a better posterior in the same amount of time as the MCMC. The performance of the SNPE algorithm can be further improved by optimizing the Python code through techniques such as Numba or writing the back-end algorithm in C++, however, these are beyond the scope of this paper. The results indicate that SNPE posterior is not perfect, however, it still assigns a significant probability mass to the true data-generating process and in some cases, even concentrates around the true parameters. This is a marked improvement compared to the MCMC algorithm, which even after 10 million iterations, generally only produces highly concentrated posteriors along a small range that often does not include the true data-generating parameter.

IV.G. HANK Model

The fifth model is the 10 parameter HANK model with the parameters discussed in the model section:

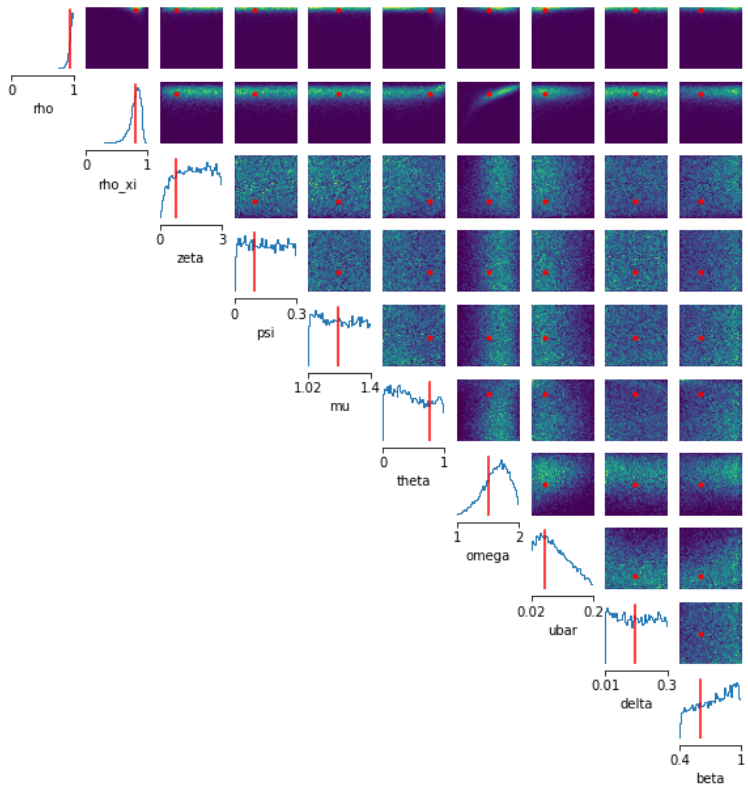

Figure XVII: Ten Parameter HANK model–Partially Observed

The estimation of the HANK model is the focus of the next discussion. The HANK model features two observable shocks, namely the unemployment rate and the interest rate, and thus only two observable equations and was solved with Reiter's method (Reiter, 2009). Despite having a large latent state space of 810 latent variables, the estimation of the model was carried out successfully removing the need for lossy dimensionality reduction of Ahn et al. (2018). Dimensionality reduction is needed when evaluating the likelihood as filtering rapidly runs into the curse of dimensionality. However, my approach does not require likelihood evaluations and by extension the

intermediate filtering step. As one can see, the approach used in the estimation process was able to identify the parameters with confidence, and the estimated posteriors agrees with the ground truth. This demonstrates the feasibility of estimating HANK models with a large latent state space, which is even computationally infeasible for an approach with a likelihood without using dimension reduction.

The approach used in the estimation process was able to estimate the parameters ρ, ρ_ξ, ω and $\bar{u}$ with high accuracy. The mode of the distribution of these parameters was found to be either at the true parameter value or in the case of ω, close to the true parameter value. However, for the other parameters, the likelihoods were found to be flat, which may be attributed to the limited impact of these parameters on the chosen observed variables. In conclusion, the estimation of the HANK model showed promising results, and highlights the potential of the approach used in the estimation process to estimate models with large latent state spaces.

An alternative method for HANK estimation is Kase, Melosi and Rottner (2022). My approach has the advantage of being developed for over five years in the machine learning space, while their appraoch seems novel. In his HANK application, they uses about 100 thousand samples to train their neural network to reproduce the likelihood of a 5 parameter model. With the Smets and Wouters (2007) section (see IV.F.) I trained a 36 parameter model with 500 thousand iterations and below in the Winberry section (see IV.H. I discuss training a 3 parameter model with 10k samples. While they later call the neural net many more time to get MCMC to converge, my method seems to converge roughly as fast as the speed required to train the neural net they use to reproduce the likelihood. Another issue with his approach as it currently stands is that in his application, he uses tight priors with modes precisely at the true parameter value. This even if the likelihood was not informative, his approach would recover the true parameter values. At the same time my approach uses uniform priors which are diffuse, and still often recover the true data generating process at my posterior modes. As an approach that uses the likelihood for estimation, clearly their approach has advantages when there is a likelihood, however my approach can estimate likelihood intractable models which their approach can't without some sort of modification.

IV.H. Winberry Heterogenous Agent Model

A sixth model is a Winberry (2018) approach to the Krusell and Smith (1998) model with full micro-moments information as implemented in Liu and Plagborg-Møller (2023). My approach recovers similar posterior results as Liu and Plagborg-Møller (2023), but my approach is around 13x faster per iteration as I only need micro and macro variable samples, without having the reconstruct the likelihood and use a sampling approach to numerically approximate an integral. I use the same number of iterations (10000) as Liu and Plagborg-Møller (2023), and seems to converge to the posterior like Liu and Plagborg-Møller (2023). I attempted to estimate even more parameters, however, the results suggest the model is unidentified, so I did not show my results. Liu and Plagborg-Møller (2023) do not show estimation results with more parameters.

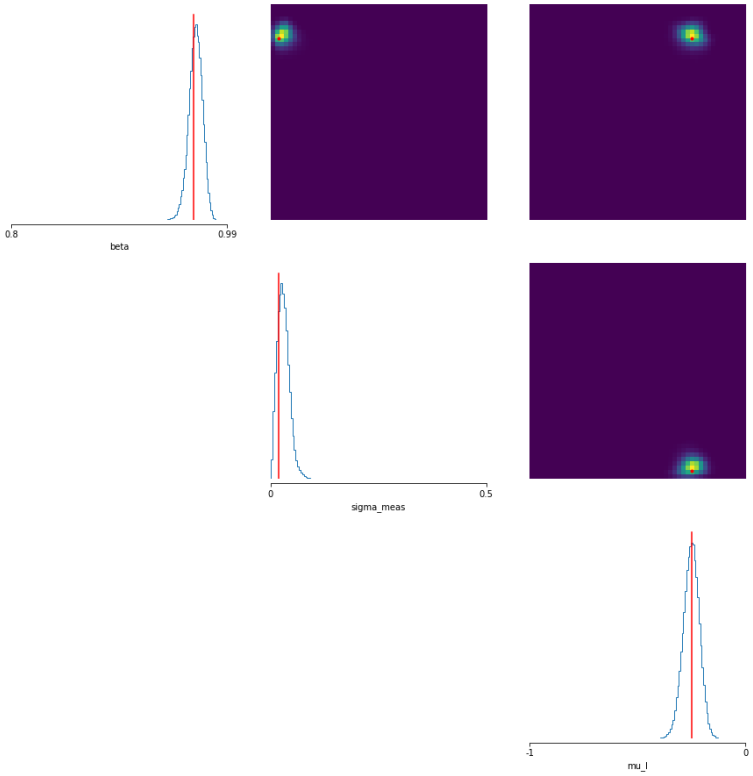

Figure XVIII: SBI Results

Here are the Liu and Plagborg-Møller (2023) results:

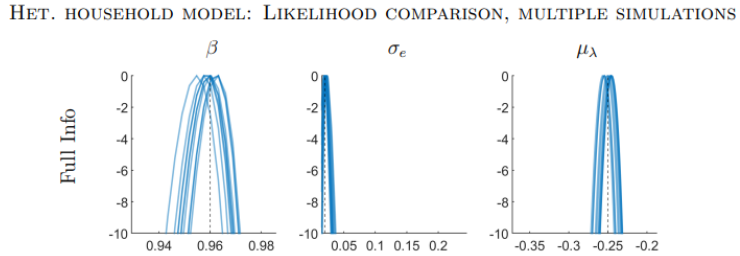

Figure XIX: Liu and Plagborg-Moller Results

The results of the study are presented in the form of a figure above. The figure shows that the posterior concentrates almost entirely around the true parameter value, suggesting that the estimation was highly informative regarding the parameter value and accurately reproduces the simulation parameters. The results are compared to those obtained by Liu and Plagborg-Møller (2023), which are presented in above as well. Both sets of results are similar and demonstrate the viability of the approach used in this study. This example illustrates how SNPE is a fast and reliable approach to the estimation of heterogeneous agent macroeconomic models.

IV.I. Empirical Application

Additionally, using the Winberry (2018) solution method, and extending Liu and Plagborg-Møller (2023), I estimated Krusell and Smith (1998) model on empirical data using both micro wealth percentiles, and macro variables: output, investment, consumption and wage. The estimation routine results in posteriors that are somewhat diffuse. However, since SNPE produces more reproducible posteriors, even when they are flat, the model can still be reproduced. The posterior variables estimated are β: the discount rate; μ_l: the labor productivity heterogeneity parameter; measurement error for each of the 4 macro variables and the micro wealth process (meas_error 5); and a lower bound on net borrowing.

The macro data comes from standard sources (FRED etc.). The micro data comes from the total household net worth variable in the Survey of Income and Program Participation (SIPP). One challenge is that not all years are recorded, which would require, if using MCMC, modification

of the loss function depending on the presence of data. However for my approach, I only need to produce simulated micro data for the years with observed data, and the model automatically matches simulated data with real data using the surrogate conditional density function posterior estimator. Another problem with the SIPP data is the number of people interviewed in the years with interviews, range form 700 people to 1700 people. A common way to deal with this is to match 100 samples with a 100 random sample from the data. This throws away a information. Due to the simulation advantages of SNPE, I can match percentiles transforming the empirical data into the first to 99th percentiles, and matching that with model generated percentiles. This provides more information and implicitly use the entire micro data set to discipline the model. If one wanted to match percentiles with a likelihood approach, one would have to calculate 99 order statistics, which is relatively intractable. However, the downside of this approach is that one can no longer determine the joint relationship of wealth and employment. The only result I can think of is to sum employment across agents to form a macro unemployment rate. Here are the results of the empirical exercise:

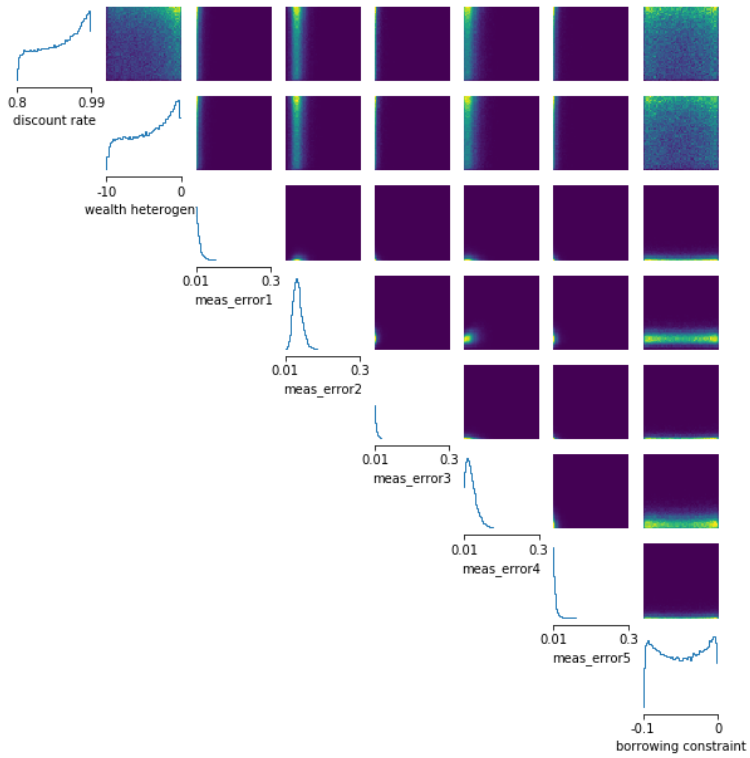

Figure XX: Krussel Smith/Winberry Empirical Results

The parameters of interest are all somewhat diffuse, although the modes make sense. The discount rate is close to .99, the labor productivity parameters is a labor productivity level (λ) which has expectation of 1 and $E[log\lambda] = \mu_\lambda$ these two relationships combine to determine the standard deviation (heterogeneity) of the productivity process. Liu and Plagborg-Møller (2023) point out that a μ_λ of -.25 is in line with Piketty, Saez and Zucman (2018), which is roughly where the mode of the posterior is. The empirical exercise suggests this model has decent, but not exceptional identification of the heterogeneous parameter of interest, λ.

V. Conclusion

In this paper, I introduce a novel approach to estimate models in a Bayesian or MLE framework, which does not require a likelihood function and also derives posteriors faster and more accurately

than baseline MCMC and other competing methods. The model can Bayesian estimate, relatively large models like the 36 parameter Smets and Wouters (2007). The method has also been applied to HANK models with large latent state spaces and has been successful in providing credible posterior estimates faster than baseline methods. Moreover, it can be applied to solution methods such as value function iteration and projection, which do not yield a likelihood function. This approach can be applied as broadly as MSM with improved guarantees for efficiency and posterior estimation. The method can take in any solution method and can estimate any black box model as long as the model can produce simulated data.

The proposed method has a wider range of applications beyond macroeconomics. It can be used in the fields of industrial organization, applied game theory, and econometrics. In industrial organization and game theory, the method can be used to solve structural and dynamic models. In econometrics, it can be applied to estimate general latent time series models effectively and efficiently, without the need for a likelihood function. Additionally, the the suite of conditional density estimators can be used for density estimation in high dimensional settings.

In conclusion, the proposed method has a much broader scope of application than just macroeconomic modeling. The applications of this technique are not limited to this paper, and have the potential to be explored in greater depth in future research.

References

Adjemian, Stéphane, Houtan Bastani, Michel Juillard, Fréderic Karamé, Ferhat Mihoubi, Willi Mutschler, Johannes Pfeifer, Marco Ratto, Normann Rion, and Sébastien Villemot. 2022. "Dynare: Reference Manual Version 5." CEPREMAP Dynare Working Papers 72.

Ahn, SeHyoun, Greg Kaplan, Benjamin Moll, Thomas Winberry, and Christian Wolf. 2018. "When inequality matters for macro and macro matters for inequality." *NBER macroeconomics annual*, 32(1): 1–75.

Andrews, Isaiah, and Anna Mikusheva. 2015. "Maximum likelihood inference in weakly identified dynamic stochastic general equilibrium models." *Quantitative Economics*, 6(1): 123–152.

Auclert, Adrien, Bence Bardóczy, Matthew Rognlie, and Ludwig Straub. 2021*a*. "Using the sequence-space Jacobian to solve and estimate heterogeneous-agent models." *Econometrica*, 89(5): 2375–2408.

Auclert, Adrien, Hannes Malmberg, Frederic Martenet, and Matthew Rognlie. 2021*b*. "Demographics, wealth, and global imbalances in the twenty-first century." National Bureau of Economic Research.

Barron, Andrew R. 1993. "Universal approximation bounds for superpositions of a sigmoidal function." *IEEE Transactions on Information theory*, 39(3): 930–945.

Bishop, Christopher M. 1994. "Mixture density networks."

Caballero, Ricardo J, and Eduardo MRA Engel. 2007. "Price stickiness in Ss models: New interpretations of old results." *Journal of monetary economics*, 54: 100–121.

Caglio, Cecilia R, R Matthew Darst, and Sebnem Kalemli-Özcan. 2021. "Risk-taking and monetary policy transmission: Evidence from loans to smes and large firms." National Bureau of Economic Research.

Chewi, Sinho. 2023. "Log Concave Sampling."

Childers, David, Jesús Fernández-Villaverde, Jesse Perla, Christopher Rackauckas, and Peifan Wu. 2022. "Differentiable State-Space Models and Hamiltonian Monte Carlo Estimation." National Bureau of Economic Research.

Dixit, Avinash K, and Joseph E Stiglitz. 1977. "Monopolistic competition and optimum product diversity." *The American economic review*, 67(3): 297–308.

Draxler, Felix, Christoph Schnörr, and Ullrich Köthe. 2022. "Whitening Convergence Rate of Coupling-based Normalizing Flows." *Advances in Neural Information Processing Systems*, 35: 37241–37253.

Duffie, Darrell, and Kenneth J Singleton. 1990. "Simulated moments estimation of Markov models of asset prices."

Durkan, Conor, Iain Murray, and George Papamakarios. 2020. "On contrastive learning for likelihood-free inference." 2771–2781, PMLR.

Farrell, Max H, Tengyuan Liang, and Sanjog Misra. 2018. "Deep neural networks for estimation and inference: Application to causal effects and other semiparametric estimands." *arXiv preprint arXiv:1809.09953*.

Gallant, A Ronald, and George Tauchen. 1996. "Which moments to match?" *Econometric theory*, 12(4): 657–681.

Glöckler, Manuel, Michael Deistler, and Jakob H Macke. n.d.. "Variational methods for simulation-based inference."

Goodfellow, Ian, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. "Generative adversarial nets." *Advances in neural information processing systems*, 27.

Greenberg, David, Marcel Nonnenmacher, and Jakob Macke. 2019. "Automatic posterior transformation for likelihood-free inference." 2404–2414, PMLR.

Haan, Peter, and Arne Uhlendorff. 2006. "Estimation of multinomial logit models with unobserved heterogeneity using maximum simulated likelihood." *The Stata Journal*, 6(2): 229–245.

Hajivassiliou, Vassilis A, et al. 1997. *Some practical issues in maximum simulated likelihood.* Suntory-Toyota International Centre for Economics and Related Disciplines ....

Hennessy, Christopher A, and Toni M Whited. 2007. "How costly is external financing? Evidence from a structural estimation." *The Journal of Finance*, 62(4): 1705–1745.

Herbst, Edward, and Frank Schorfheide. 2014. "Sequential Monte Carlo sampling for DSGE models." *Journal of Applied Econometrics*, 29(7): 1073–1098.

Herbst, Edward P, and Frank Schorfheide. 2015. *Bayesian estimation of DSGE models.* Princeton University Press.

Hermans, Joeri, Volodimir Begy, and Gilles Louppe. 2020. "Likelihood-free mcmc with amortized approximate ratio estimators." 4239–4248, PMLR.

Hoffman, Matthew D, Andrew Gelman, et al. 2014. "The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo." *J. Mach. Learn. Res.*, 15(1): 1593–1623.

Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. 1989. "Multilayer feedforward networks are universal approximators." *Neural networks*, 2(5): 359–366.

Huang, Chin-Wei, David Krueger, Alexandre Lacoste, and Aaron Courville. 2018. "Neural autoregressive flows." 2078–2087, PMLR.

Kaji, Tetsuya, Elena Manresa, and Guillaume Pouliot. 2020. "An adversarial approach to structural estimation." *arXiv preprint arXiv:2007.06169.*

Kase, Hanno, Leonardo Melosi, and Matthias Rottner. 2022. "Estimating nonlinear heterogeneous agents models with neural networks."

Keane, Michael P, and Kenneth I Wolpin. 1994. "The solution and estimation of discrete choice dynamic programming models by simulation and interpolation: Monte Carlo evidence." *the Review of economics and statistics*, 648–672.

Khan, Aubhik, and Julia K Thomas. 2008. "Idiosyncratic shocks and the role of nonconvexities in plant and aggregate investment dynamics." *Econometrica*, 76(2): 395–436.

Kristensen, Dennis, and Yongseok Shin. 2012. "Estimation of dynamic models with nonparametric simulated maximum likelihood." *Journal of Econometrics*, 167(1): 76–94.

Krusell, Per, and Anthony A Smith, Jr. 1998. "Income and wealth heterogeneity in the macroeconomy." *Journal of political Economy*, 106(5): 867–896.

Lerman, Steven, and Charles Manski. 1981. "On the use of simulated frequencies to approximate choice probabilities." *Structural analysis of discrete data with econometric applications*, 10: 305–319.

Liu, Laura, and Mikkel Plagborg-Møller. 2023. "Full-information estimation of heterogeneous agent models using macro and micro data." *Quantitative Economics*, 14(1): 1–35.

McFadden, Daniel. 1989. "A method of simulated moments for estimation of discrete response models without numerical integration." *Econometrica: Journal of the Econometric Society*, 995–1026.

Mohammad-Djafari, Ali, and Hacheme Ayasso. 2009. "Variational Bayes and mean field approximations for Markov field unsupervised estimation." 1–6, IEEE.

Nathoo, FS, ML Lesperance, AB Lawson, and CB Dean. 2013. "Comparing variational Bayes with Markov chain Monte Carlo for Bayesian computation in neuroimaging." *Statistical methods in medical research*, 22(4): 398–423.

Ottonello, Pablo, and Thomas Winberry. 2020. "Financial heterogeneity and the investment channel of monetary policy." *Econometrica*, 88(6): 2473–2502.

Pakes, Ariel, and David Pollard. 1989. "Simulation and the asymptotics of optimization estimators." *Econometrica: Journal of the Econometric Society*, 1027–1057.

Papamakarios, George, and Iain Murray. 2016. "Fast ε-free inference of simulation models with bayesian conditional density estimation." *Advances in neural information processing systems*, 29.

Piketty, Thomas, Emmanuel Saez, and Gabriel Zucman. 2018. "Vol. 133 May 2018 Issue 2." *The Quarterly Journal of Economics*, 553: 609.

Reiter, Michael. 2009. "Solving heterogeneous-agent models by projection and perturbation." *Journal of Economic Dynamics and Control*, 33(3): 649–665.

Rendahl, Pontus. 2017. "Linear time iteration." IHS Economics Series.

Rubin, Donald B. 1984. "Bayesianly justifiable and relevant frequency calculations for the applied statistician." *The Annals of Statistics*, 1151–1172.

Schmitt-Grohé, Stephanie, and Martın Uribe. 2004. "Solving dynamic general equilibrium models using a second-order approximation to the policy function." *Journal of economic dynamics and control*, 28(4): 755–775.

Siarry, Patrick, Gérard Berthiau, François Durdin, and Jacques Haussy. 1997. "Enhanced simulated annealing for globally minimizing functions of many-continuous variables." *ACM Transactions on Mathematical Software (TOMS)*, 23(2): 209–228.

Smets, Frank, and Rafael Wouters. 2007. "Shocks and frictions in US business cycles: A Bayesian DSGE approach." *American economic review*, 97(3): 586–606.

Wainwright, Martin J, and Michael Irwin Jordan. 2008. *Graphical models, exponential families, and variational inference.* Now Publishers Inc.

Winberry, Thomas. 2018. "A method for solving and estimating heterogeneous agent macro models." *Quantitative Economics*, 9(3): 1123–1151.

A Bayesian Estimation in General and Metropolis-Hastings

The predominant technique for performing Bayesian inference in economics is to perform MCMC (Herbst and Schorfheide, 2015). In MCMC, one only concerns oneself with relative jumps between parameter values. For example, if one knows both the prior and the likelihood for any given point, $p(X|\theta)p(\theta)$, which is assumed to be known, one knows the relative likelihood of being in any point versus any other. One point in parameter space may be twice as likely to be visited in the posterior as another point, even if the actual probabilities are not known, because the normalization term in Bayes rule, $P(x)$ is not known. Since one knows the relative probabilities, one can design a random walk so that a walker visits probabilities equivalent to the ratios of the relative probabilities. One way to do that is if a walker can choose to compare any point in the posterior to the point the walker is currently at, the walker will move to the second point with the likelihood according to the ratio of the probabilities. If the second point is half as likely to be in the posterior, the walker will move with probability one half and probability one half stay in place. If the second point is more likely to be in the posterior, say twice as likely, the walker will move to the second point for sure,

with the understanding that if the walker randomly selected to move back, the move back would be now with half the probability.

In this way, the walker moves across the space of the posterior with relative frequency according to $P(x|\theta)P(\theta)$ ratio between points, which ultimately means that one is sampling from $P(\theta|x)$ without calculating the normalizing partition function. In practice, MCMC is a little more complicated, as often the parameter space is unbounded, and one can't sample the entire space with equal probability. Then this involves the use of a proposal distribution and the use of importance sampling, where one samples from closer points with higher probability, but the likelihood is normalized to take into account the fact that nearer points are more likely to be selected than further points. This allows MCMC to sample in the long run from the entire space as long as one adjusts for the probability that the proposal distribution is more likely to sample from one. In particular call $P(x|\theta)P(\theta)$ as the unnormalized posterior value for a point θ and $P(x|\theta')P(\theta')$ is the unnormalized posterior for a point θ'. Given a proposal distribution $f(\theta)$ which is the probability of visiting θ from θ' and $f(\theta')$ as the probability of visiting θ' from θ under this proposal distribution, then the equation that preserves the relative ratios and samples from the posterior will move to the new point if:

$$\frac{\frac{P(x|\theta)P(\theta)}{f(\theta)}}{\frac{P(x|\theta')P(\theta')}{f(\theta')}}$$

Since often the f distribution is assumed to be symmetric, the f terms drop out leaving the ratios of the unnormalized probabilities used to decide when to walk. Unsurprisingly this approaches is quite slow as one has to visit the whole space sequentially with higher probability points being visited many more times than lower probability points. Alternatives like variational inference have been proposed which speed up the inference at the cost of some bias (Wainwright and Jordan, 2008), but it is beyond the scope of this paper. The approach I propose, SNPE, does not require knowing the likelihood, something that may be intractable to calculate, depending on the solution algorithm for macroeconomic models.

AA. Feed Forward Neural Network

This section will describe the feed forward neural network. At there core, feed forward neural networks are compositions of vector valued linear equations with non-linearities between each composition. For example a neural network with a single hidden layer could be constructed:

$$Y = W_2 * \sigma(W_1 * X + b_1) + b_2 \tag{7}$$

Here X is some vector valued input with dimensionality n by assumption. W_1 is a k by n matrix that turns a n dimensional input into a k dimensional basis for the second layer. Likewise b_1 is a k dimensional intercept term. W_2 turns the k dimensional basis vector from the first linear regression into the m dimensional output vector and b_2 is the m dimensional intercept term. Since W_1, W_2, b_1, b_2, are all linear operations, arbitrary compositions of them will still be linear. However if you add a nonlinearity term, σ between the compositions, with loose regularity conditions, a single hidden layer neural network as shown above can approximate any continuous function (Hornik, Stinchcombe and White, 1989), as the k dimensional basis expands to infinity. The non-linearity can be almost any function but for the universal approximator proofs clearly has to have an image that is the real line. The one I use is the Leaky ReLU:

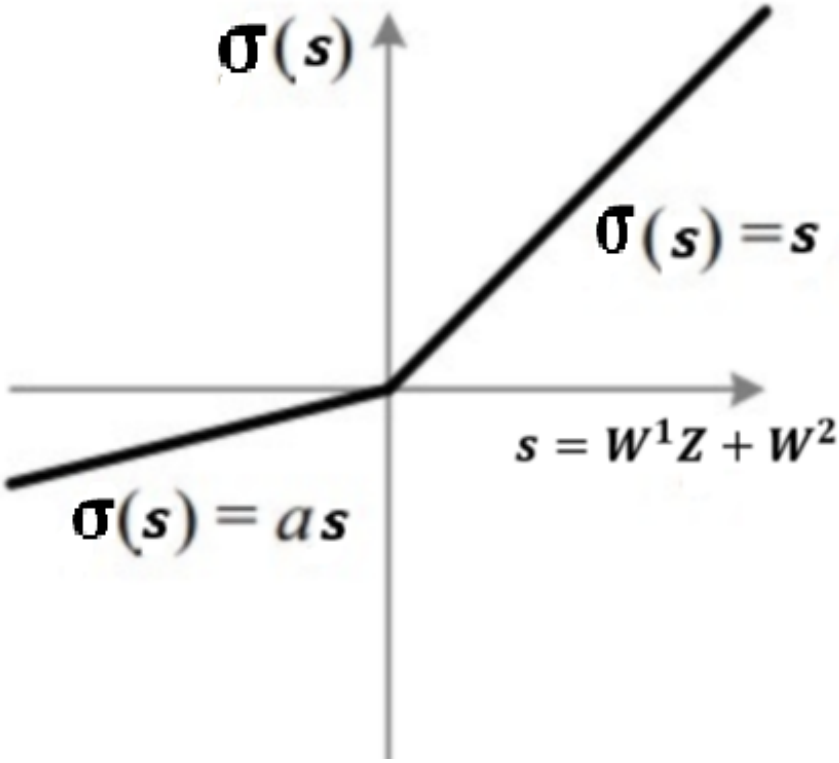

Figure XXI: Feed Forward Neural Network

Other non-linearities include Logits, hyperbolic tangents, ReLUs, and ELUs which are modifications of the Leaky ReLU. Additionally, one can make deeper neural networks by composing for example, W_3, and b_3, more vector linear regressions with non-linearities. Here is a graphical representation of layers in a neural network stacked one on top of another:

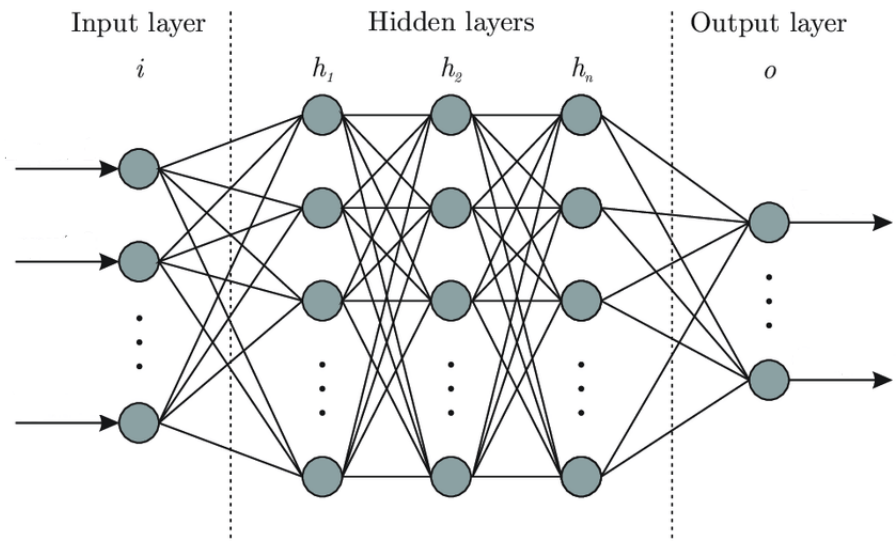

Figure XXII: Feed Forward Neural Network

In this case, there are three hidden layer in combination with the input vector and the output vector. These layers can be arbitrarily wide, with width and depth associated with increased flexibility but also increased parameter count for risk of over-fitting.

B Economic Models

BA. RBC Model

In the present article, the results of estimating the RBC model using the SNPE algorithm are presented and discussed. The RBC model is a standard model in macroeconomics and it has four parameters, α, β, δ, and ρ, with a fixed CRRA elasticity of 2. The first 100 iterations are dropped to attain a steady-state behavior. This RBC model is standard and comes from Alisdair McKay's note on heterogeneous agent models: `https://alisdairmckay.com/Notes/HetAgents/index.html`.

The utility function is a standard CRRA utility:

$$u(C_t) = E \sum_t \beta^t \frac{C_t^{1-\gamma}}{1-\gamma} \tag{8}$$

The production function is standard Cobb-Douglas with productivity:

$$Y_t = Z_t K_t^{\alpha} \bar{L}^{1-\alpha} \tag{9}$$

Where $\bar{L}$ is fixed labor and Z_t is productivity and K_t is capital which evolves according to a standard law of motion:

$$K_t = (1-\delta)K_{t-1} + Y_t - C_t \tag{10}$$

Productivity evolves according to a AR(1) process in logs:

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{11}$$

This model is solved via time iteration (Rendahl, 2017) and matches consumption, investment, productivity to real data, interest rates and capital are the latent unobserved variables. This is done on data both synthetic and real.

The results of the posterior are displayed below, where the red bar represents the true parameter values and the blue line represents the posterior as estimated by the simulation-based inference approach:

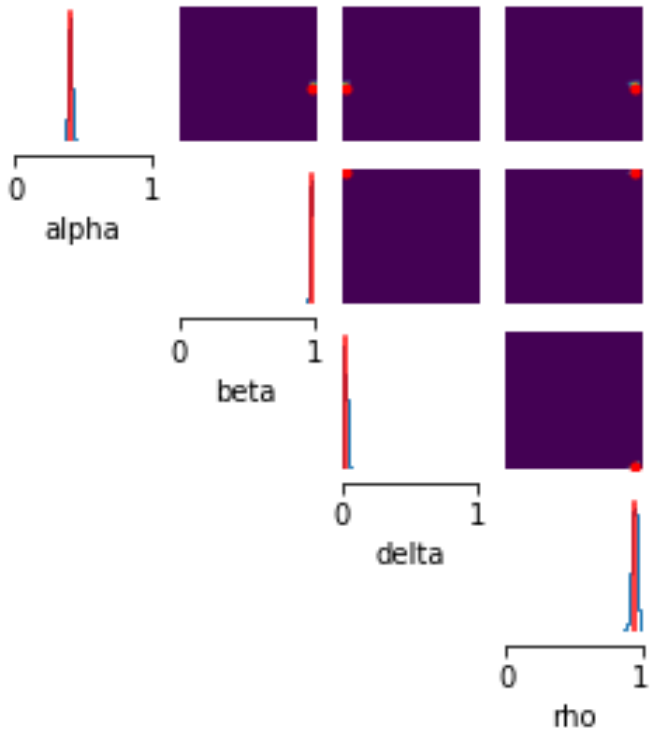

Figure XXIII: Simulated Data RBC

The diagonal density plots in the upper triangular portion of the graph display two-way densities, with the color indicating the probability mass. The approximate ground truth is obtained by running the MCMC for 2 million iterations as show below:

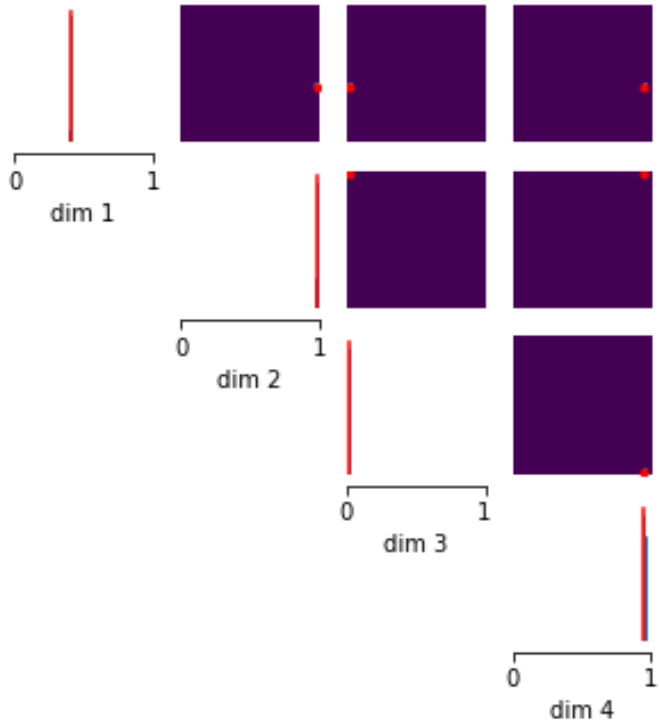

Figure XXIV: Simulated Data RBC

Below the same RBC model is also estimated on real data, where there is no available ground truth. The parameters derived from the real data are not necessarily realistic, which is expected as the RBC model is a simple model. The method is uncertain about the ground truth, as indicated by the spread of the posterior. The model also predicts some parameters that are not in line with theory, such as the discount rate, β, and the parameter on the productivity process, ρ. The depreciation rate, δ, is the only parameter that is relatively reasonable. Below is the posterior estimated on real data:

In conclusion, the SNPE algorithm is applied to estimate the RBC model and the results are displayed and discussed. The results indicate that the SNPE algorithm can estimate the RBC model to a good degree of accuracy. The model's limitations in real-world applications are also highlighted, as it fails to estimate some parameters that are not in line with theoretical predictions.

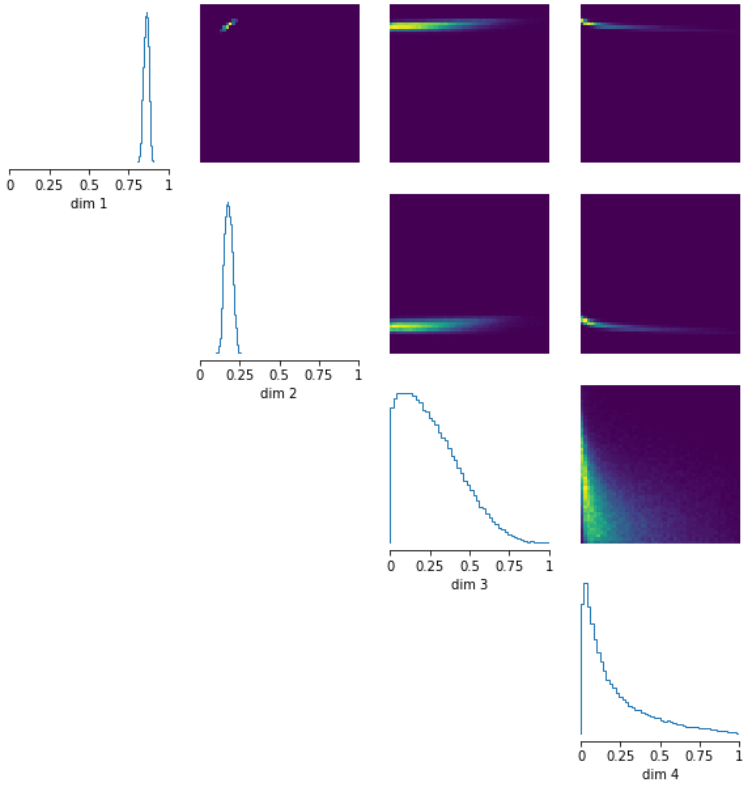

Figure XXV: Real Data RBC

This highlights the need for more complex models to better reflect real-world economic scenarios, which will be illustrated in the next models.

BB. Cash Flow Model

in this section, I am estimating a partial equilibrium model of firm cash flow by utilizing value function iteration. The model is specified with four parameters, α, δ, σ, and ρ. α is the capital share in the production function. δ is depreciation rate of capital. σ is the standard deviation of log productivity AR process. ρ is the slope of the log productivity AR process. The interest rate was set at 5 percent and the discount factor was calculated as 1/(1+r).

This model is a straightforward infinite horizon partial equilibrium model with risk neutral managers choosing investment to maximize the present value of a stream of firm cash flows and comes from solution code written by Emil Lakkis and Toni Whited.

Utility/cash flows are give by this formula:

$$U(K_t, I_t, Z_t) = \sum_t \beta^t * (\pi(K_t, Z_t) - I_t) \tag{12}$$

Where K_t, Z_t, and I_t are the same as in the RBC model–capital, productivity and investment respectively. π is the profit function.

The capital law of motion is like the RBC model:

$$K_t = (1 - \delta)K_{t-1} + I_t \tag{13}$$

Productivity evolves according to a AR(1) process in logs:

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{14}$$

Profit is defined as:

$$\pi(K_t, Z_t) = Z_t K_t^{\alpha} \tag{15}$$

This model shares similarities with the RBC model, with the main difference being the utility function and the value function solution method. This estimation method matches capital levels generated by a parameterized model.

To estimate the model parameters, two different estimation methods were applied, including a simulation-based inference approach using SNPE and SNVI, which is mentioned in the paper body but discussed further in the CD. section in the appendix. From the visual inspection of the charts, it can be observed that the posteriors obtained from the two estimation methods are similar, suggesting that the same distribution was learned. Additionally, the mode of the distribution appears to be close to the actual parameter values, which indicates that the model is capable of accurately recovering the true parameters.

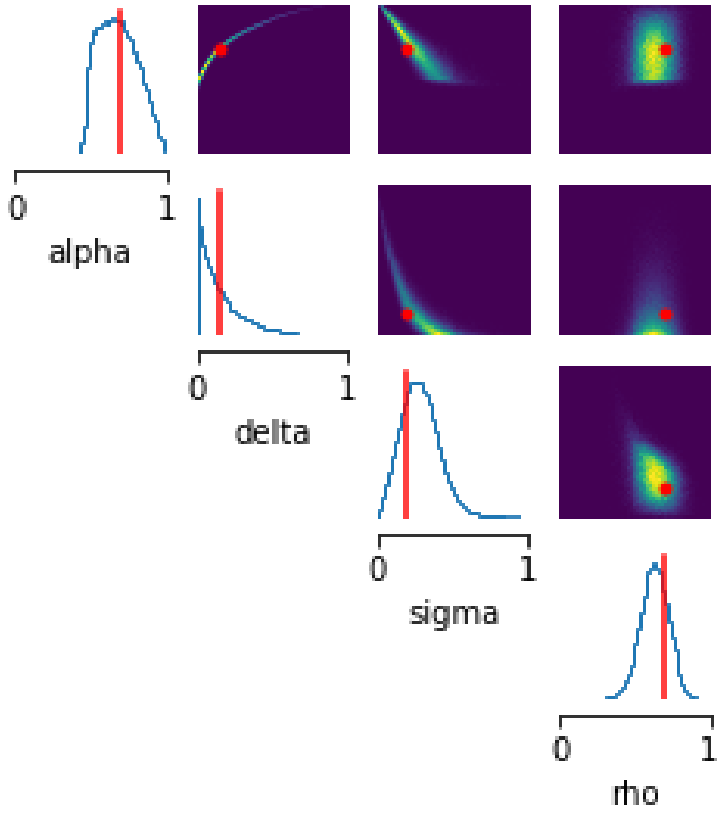

Figure XXVI: Value Function Iteration with a Normalizing Flow Density Estimator

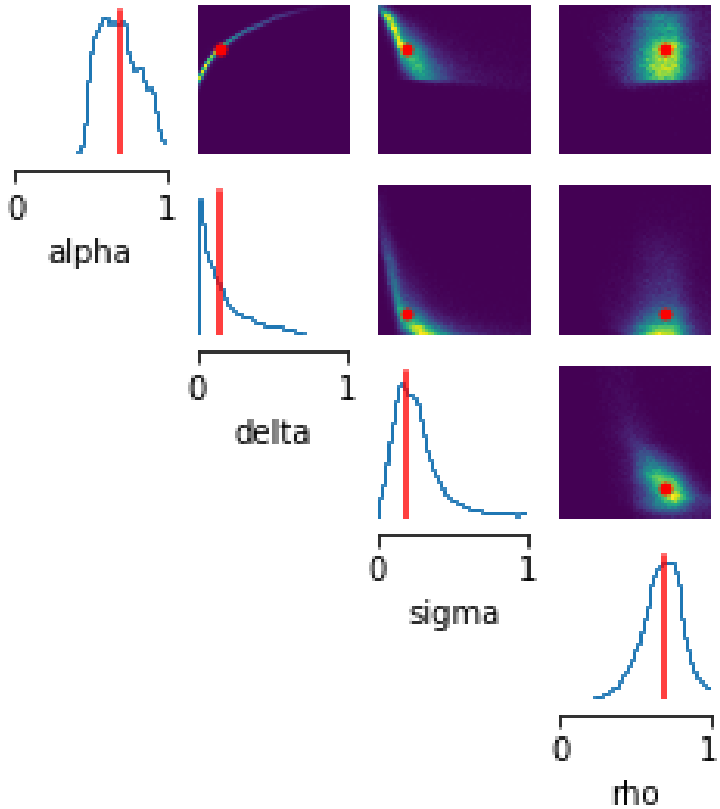

Figure XXVII: Value Function Iteration with a GAN Density Estimator

BC. Lucas Asset Pricing Projection Model

In the following section, we demonstrate the estimation procedure on a Lucas asset pricing model, which is solved via projection without measurement error. Following Mohammed Ait Lahren's code (`https://notes.quantecon.org/submission/5f44808ad24fdb001162a53b`), each agent maximizes CRRA utility:

$$max_i E_t[\beta^t \frac{c_t^{1-\gamma} - 1}{1-\gamma} \tag{16}$$

with γ set to 3. The law of motion is:

$$k_{t+1} = (1 + d_t/P_t)k_t - c_t/P_t \tag{17}$$

The state of the system is defined as dividends received plus price of the amount of assets owned:

$$m_{t+1} = (P_{t+1} + d_{t+1})k_{t+1} \tag{18}$$

Putting these three equations together one gets the pricing kernel:

$$p_t = \beta^t E_t[(\frac{c_{t+1}}{c_t})^{-\gamma}(d_{t+1} + p_{t+1})] \tag{19}$$

Dividends evolve under an $AR(1)$ process:

$$d_{t+1} = \mu_d + \rho_d * d_t + \epsilon \tag{20}$$

Where ϵ is a shock with standard deviation σ. When behaving optimally, consumption also equals the dividends received. This model matches the expected return and risk free rate at a variety of dividend levels, which demonstrates the ability of this model to not only estimate on models solved via projection, but also to work on cross sectional problems as well.

The purpose of this estimation is to showcase the capability of the SNPE approach to estimate models of this nature:

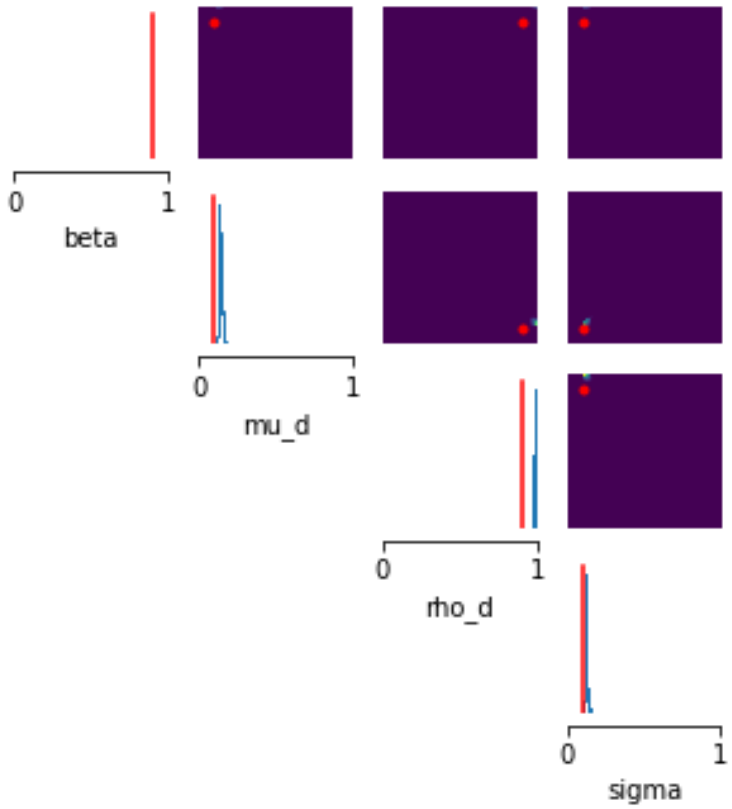

Figure XXVIII: Projection Model

This estimation procedure took under 10 hours to complete on an 8-core Intel i7 machine, highlighting the computational efficiency of this approach. This result also demonstrates that models solved via projection can be effectively estimated without measurement error using the SNPE framework. The posterior accuracy is pretty good, however it seems like the posteriors have a mode that is slightly shifted and while close to the true parameter value, do not put much support the on true value along the μ_d and ρ_d dimensions.

BD. The Value Function Iteration Bequests Model

I estimate a model of bequests over time. The model illustrates the benefit of my approach as it cannot be estimated by perturbation because of the kink at the intersection of the consumption

versus adjusting illiquid savings intersection. Thus one has to use simulation-based inference to Bayesian estimate this model solved via value function iteration.

This model comes from Jeppe Druedahl's open source model example: `https://github.com/pkofod/vfi/blob/master/Fast%20VFI.ipynb`. The utility function is:

$$u(b_t, c_t) = \frac{[\phi(b_t + \underline{b})^{1-\gamma} + (1-\phi)c_t^{1-\gamma}]^{\frac{1-\rho}{1-\gamma}}}{1-\rho} \tag{21}$$

Here b_t is bequest or non-liquid savings. c_t is consummation. b, ϕ, γ, ρ are all parameters. The bequest utility function (end of life utility) is:

$$\nu(a_t, b_t) = \varphi \frac{(a_t + b_t + \underline{q})^{1-\vartheta}}{1-\vartheta} \tag{22}$$

Here a_t is liquid savings and q, θ, φ are all parameters. The value function is give by:

$$v_t(m_t, n_t, l_t) = \max\{v_t^{keep}(m_t, n_t, l_t), v_t^{adj}(m_t, n_t, l_t)\} \tag{23}$$

$$\text{s.t.} \tag{24}$$

$$x_t = m_t + (1-\tau)n_t \tag{25}$$

Here m_t is beginning of period liquid savings (analogous to end of period a_t) and n_t is beginning of period illiquid savings (analogous to end of period b_t). x_t is the pooled value of total liquid and

illiquid savings. l_t is labor income. The post-decision value function is:

$$w_t(a_t, b_t, l_t) = \nu(a_t, b_t), t = T \tag{26}$$

$$w_t(a_t, b_t, l_t) = \mathbb{E}_t \left[\max\{v_t^{keep}(m_{t+1}, n_{t+1}, l_{t+1}), v_t^{adj}(x_{t+1}, l_{t+1})\} \right], t < T \tag{27}$$

$$\text{s.t.} \tag{28}$$

$$l_{t+1} \sim F(l_t) \tag{29}$$

$$m_{t+1} = (1 + r_a)a_t + \omega l_{t+1} \tag{30}$$

$$n_{t+1} = (1 - \delta)b_t \tag{31}$$

$$x_{t+1} = m_{t+1} + (1 - \tau)n_{t+1} \tag{32}$$

Here labor income comes from a distribution $F(l_t)$. Likewise r_a, ω, δ, τ are all parameters. The keep value function is:

$$v_t^{keep}(m_t, n_t, l_t) = \max_{c_t \in [0, m_t]} u(n_t, c_t) + \beta w_t(a_t, b_t, l_t) \tag{33}$$

$$\text{s.t.} \tag{34}$$

$$a_t = m_t - c_t \tag{35}$$

$$b_t = n_t \tag{36}$$

The keep value function is the value function at a point t if the agent chooses to consume and not withdraw or deposit funds into the illiquid asset, b_t. The adjust value function is:

$$v_t^{adj}(x_t, l_t) = \max_{b_t \in [0, x_t]} v_t^{keep}(m_t, n_t, l_t) \tag{37}$$

$$\text{s.t.} \tag{38}$$

$$m_t = x_t - b_t \tag{39}$$

$$n_t = b_t \tag{40}$$

The adjust value function is the value of adjusting the illiquid asset. Notice the agent is unable to

consume this period. This model cannot be effectively modeled with a method like perturbation, because of its nonlinear and kinked decision rule. Even so, I use my estimation technique to get a Bayesian posterior distribution. This model matches the liquid asset, illiquid asset and consumption to synthetic data generated by a model.

BE. Smets-Wouters 2007 Model

A description of the Smets and Wouters (2007) discussed in the results: This section will mainly illustrate the equations and the parameters so one can cross reference parameters shown in the diagram with the correct equation in the DSGE model. This model comes from the paper Smets and Wouters (2007). More details on the economics of the equations can be found there. This model can be estimated both with MCMC and SNPE. Thus this model mostly illustrates the increases in posterior accuracy when using an uninformative prior which goes hand in hand with estimation speed when using SNPE vs MH-MCMC. Here is a reference for all the Smets-Wouter abbreviations and the actual variable name, written in order of appearance in the **?** picture of marginal posteriors distributions contained in the results section.

calfa = Capital Share

csigma = Elasticity of Consumption

cfc = One Plus Fixed Costs

cgy = 'Productivity Shock Effect on Government Spending

csadjcost = Capital Adjustment Cost Elasticity

chabb = Habit Formation

cprobw = Degree of Wage Stickiness

csigl = Elasticity of Labor Supply

cprobp = Degree of Price Stickiness

cindw = Wage Indexation cindp = Past Inflation Indexation

czcap = Capital Adjustment Cost Function

crpi = Taylor Rule Inflation Coefficient

crr = Taylor Rule Degree of Interest Rate Smoothing

cry = Taylor Rule Output Gap Coefficient

crdy = Taylor Rule Output Growth Gap Coefficient

crhoa = AR Term on Productivity Shocks

crhob = AR Term on Risk Premium Shocks

crhog = 'AR Term on Government Spending Shocks

crhoqs = .AR Term in Investment Shocks

crhoms = AR Term in the Taylor Rule Shock

crhopinf = AR Term in Inflation Rate Shock

crhow = AR Term in Wage Shock

cmap = Price Markup Shock to Inflation Moving Average

cmaw = Price Markup Shock to Wage Moving Average

constepinf =Trend Inflation Growth Rate

constebeta = Constant Inverse Discount Rate

constelab = Steady State Hours Worked

ctrend = Common GDP, Consumption, Investment, and Wages Trend

ea =Stdev of Productivity Shock

eb =Stdev of Risk Premium Shocks

eg = Stdev of Government Spending Shock

eqs =Stdev of Investment Shock

em =Stdev of Taylor Rule Shock

epinf = Stdev of Inflation Shock

ew = Stdev of Wage Shock

First, a brief discussion of the overview of the model. The model is a representative New Keynesian model with stickiness in both wages and prices. The model has a flexible price set of equations which determines potential GDP for monetary policy and a sticky price system with a price markup equation that determines the true economy. I will only be discussing the sticky price system of equations with an implicit understanding that there is a mirroring flexible price

system. Shocks all have auto-regressive components to more accurately match data. All equations are linearized.

The aggregate resource constraint is:

$$y_t = ccy * c_t + ciy * i_t + \epsilon_t^g + 1 * crkky * z_t \tag{41}$$

The Euler equation is given by:

$$\begin{aligned} c_t =& (chabb/cgamma)/(1 + chabb/cgamma) * c_{t-1} + (1/(1 + chabb/cgamma)) * c_{t+1} + \\ & ((csigma - 1) * cwhlc/(csigma * (1 + chabb/cgamma))) * (l_t - l_{t+1}) - \\ & (1 - chabb/cgamma)/(csigma * (1 + chabb/cgamma)) * (r_t - \pi_{t+1}) + \epsilon_t^b \end{aligned} \tag{42}$$

The investment Euler equations are:

(43)

$$i_t = (1/(1+cbetabar*cgamma))*(i_{t-1}+cbetabar*cgamma*i_{t+1}+(1/(cgamma^2*csadjcost))*q_t)+\epsilon_t^i$$

The arbitrage equation for the value of the capital stock is:

$$\begin{aligned} q_t =& - r_t + \pi_{t+1} + (1/((1 - chabb/cgamma)/(csigma * (1 + chabb/cgamma)))) * \epsilon_t^b + \\ & (crk/(crk + (1 - ctou))) * r_{t+1}^k + ((1 - ctou)/(crk + (1 - ctou))) * q_{t+1} \end{aligned} \tag{44}$$

The linearized production function is:

$$y_t = cfc * (calfa * k_t + (1 - calfa) * l_t + \epsilon_t^a) \tag{45}$$

Capital utilization:

$$k_t^s = k_{t-1} + z_t \tag{46}$$

Capital utilization is some function of the interest rate for capital:

$$z_t = (1/(czcap/(1 - czcap))) * r_t^k \tag{47}$$

Capital's law of motion is:

$$k_t = (1 - cikbar) * k_{t-1} + cikbar * i_t + cikbar * cgamma^2 * csadjcost * \epsilon_t^i \tag{48}$$

The markup equation is:

$$\mu_t^p = calfa * k_t^s + (1 - calfa) * w_t - \epsilon_t^a \tag{49}$$

This equation is the only set of economic equations that does not have a flexible price counterpart. Note that the observed variable equations also will not have a flexible price counterpart.

The New Keynesian Phillips curve:

(50)

$$\begin{aligned}\pi_t =& (1/(1 + cbetabar * cgamma * cindp)) * (cbetabar * cgamma * \pi_{t+1} + cindp * \pi_{t-1} + \\ & ((1 - cprobp) * (1 - cbetabar * cgamma * cprobp)/cprobp)/((cfc - 1) * curvp + 1) * (\mu_t^p)) + \\ & \epsilon_t^p\end{aligned}$$

The rental rate of capital equation:

$$r_t^k = w_t + l_t - k_t \tag{51}$$

The path of wages is:

(52)

$$w_t = (1/(1 + cbetabar * cgamma)) * w_{t-1} + (cbetabar * cgamma/(1 + cbetabar * cgamma)) * w_{t+1} +$$
$$(cindw/(1 + cbetabar * cgamma)) * \pi_{t-1} - (1 + cbetabar * cgamma * cindw)/(1 + cbetabar * cgamma) * \pi +$$
$$(cbetabar * cgamma)/(1 + cbetabar * cgamma) * \pi_{t+1} +$$
$$(1 - cprobw) * (1 - cbetabar * cgamma * cprobw)/((1 + cbetabar * cgamma) * cprobw) *$$
$$(1/((clandaw - 1) * curvw + 1)) * (csigl * l_t + (1/(1 - chabb/cgamma)) * c_t -$$
$$((chabb/cgamma)/(1 - chabb/cgamma)) * c_{t-1} - w_t) + \epsilon_t^w$$

The Taylor rule followed by central banks is:

(53) $$r_t = crpi * (1 - crr) * \pi_t + cry * (1 - crr) * (y_t - y_t^p) + crdy * (y_t - y_t^p - y_{t-1} + y_{t-1}^p) + crr * r_{t-1} + \epsilon_t^r$$

The observed variables equations are:

(54) $$dy_t = y_t - y_{t-1} + ctrend$$

(55) $$dc_t = c_t - c_{t-1} + ctrend$$

(56) $$di_t = i_t - i_{t-1} + ctrend$$

(57) $$dw_t = w_t - w_{t-1} + ctrend$$

(58) $$\pi_t^o = \pi_t + constepinf$$

(59) $$r_t^o = r_t + conster$$

(60) $$l_t^o = l_t + constelab$$

The autoregressive equations for shocks are:

$$\epsilon_t^a = crhoa * \epsilon_{t-1}^a + ea \tag{61}$$

$$\epsilon_t^b = crhob * \epsilon_{t-1}^b + eb \tag{62}$$

$$\epsilon_t^g = crhog * \epsilon_{t-1}^g + eg + cgy * ea \tag{63}$$

$$\epsilon_t^i = crhoqs * \epsilon_{t-1}^i + eqs \tag{64}$$

$$\epsilon_t^r = crhoms * \epsilon_{t-1}^r + em \tag{65}$$

$$\epsilon_t^p = crhopinf * \epsilon_{t-1}^p + epinfma - cmap * epinfma(-1) \tag{66}$$

$$epinfma = epinf \tag{67}$$

$$\epsilon_t^w = crhow * \epsilon_{t-1}^w + ewma - cmaw * ewma(-1) \tag{68}$$

$$ewma = ew \tag{69}$$

Some of the higher level parameters in the above equations are also related to the parameters in my diagram by the following equations:

$$cpie = 1 + constepinf/100 \tag{70}$$

$$cgamma = 1 + ctrend/100 \tag{71}$$

$$cbeta = 1/(1 + constebeta/100) \tag{72}$$

$$clandap = cfc \tag{73}$$

$$cbetabar = cbeta * cgamma^{(-csigma)} \tag{74}$$

$$cr = cpie/(cbeta * cgamma^{(-csigma)}) \tag{75}$$

$$crk = (cbeta^{(-1)}) * (cgamma^{csigma}) - (1 - ctou) \tag{76}$$

$$cw = \frac{(calfa^{calfa} * (1 - calfa)^{(1-calfa)}}{(clandap * crk^{calfa}))^{(1/(1-calfa))}} \tag{77}$$

$$cikbar = (1 - (1 - ctou)/cgamma) \tag{78}$$

$$cik = (1 - (1 - ctou)/cgamma) * cgamma \tag{79}$$

$$clk = ((1 - calfa)/calfa) * (crk/cw) \tag{80}$$

$$cky = cfc * (clk)^{(calfa-1)} \tag{81}$$

$$ciy = cik * cky \tag{82}$$

$$ccy = 1 - cg - cik * cky \tag{83}$$

$$crkky = crk * cky \tag{84}$$

$$cwhlc = (1/clandaw) * (1 - calfa)/calfa * crk * cky/ccy \tag{85}$$

$$cwly = 1 - crk * cky \tag{86}$$

$$conster = (cr - 1) * 100 \tag{87}$$

The seven observed equations dy_t, dc_t, di_t, dw_t, π_t^o, r_t^o, l_t^o, are matched using data simulated from a model with true parameters equal to the Smets and Wouters (2007) mean parameters, with slight modifications as some parameters were calibrated to 0. The MCMC appoach uses the Smets and Wouters (2007) Dynare code with small modifications like the use of a uniform prior.

BF. The HANK Reiter Model

I estimate two different HANK models. This one is solved via Reiter's method. This provides an example of estimation of models pushing the frontier in economics. The benefits of this approach allows one to estimate heterogeneous agent models in a likelihood manner without resorting to demensionality reduction. The HANK model also comes from Alisdair McKay's tutorial, which adds a heterogeneous distribution of wealth as a state variable for households combined with a relatively standard New Keynesian model.

Preferences are still CRRA, like the RBC model. The firm side is now new Keynesian with intermediate goods produced via:

$$y_{j,t} = Z_t N_{j,t} \tag{88}$$

Here Z_t is aggregate productivity and N_t is labor employed by firm j producing variety $y_{j,t}$. The final good is produced via the Dixit-Stiglitz aggregator Dixit and Stiglitz (1977):

$$Y_t = (\int_0^1 {y_{j,t}}^{\frac{\epsilon - 1}{\epsilon}})^{\frac{\epsilon}{\epsilon - 1}} \tag{89}$$

Here ϵ is the elasticity of substitution between intermediates. Output is defined as:

$$Y_t = A_t \int_0^1 n_{j,t} dj = A_t N_t \tag{90}$$

Where $n_{j,t}$ is the labor for the individual intermediate goods. Additionally from the accounting identity, output is also, $Y_t = C_t + \psi M_t H_t$ where the second term is hiring costs.

Government and bonds are defined by the following nominal and real budget constraint equa-

tions:

$$\tau P_t(w_t + d_t)(1 - u_t) + P_t B = (1 + i_{t-1})P_{t-1}B + P_t b u_t \tag{91}$$

$$\tau(w_t + d_t)(1 - u_t) + B = R_t B + b u_t \tag{92}$$

Labor market is a standard McCall Search model:

$$N_t = (1 - \delta)N_{t-1} + H_t \tag{93}$$

$$1 - u_t = (1 - \delta)(1 - u_{t-1}) + H_t \tag{94}$$

$$M_t = \frac{H_t}{u_{t-1} + \delta N_{t-1}} \tag{95}$$

$$M_t = \frac{1 - u_t - (1 - \delta)(1 - u_{t-1})}{u_{t-1} + \delta(1 - u_{t-1})} \tag{96}$$

The household maximizes the CRRA utility like the RBC model. The households budget constraint in real money is:

$$a_t + C_t = R_t * a_{t-1} + earnings \tag{97}$$

Here a_t is savings and R_t is the interest rate as determined by the bond market clearing condition. Earnings are given by:

$$\text{earnings} = \begin{cases} (1 - \tau_t)\,(w_t + d_t) & \text{if employed} \\ b & \text{if unemployed,} \end{cases} \tag{98}$$

Here w_t is wages, d_t is dividend income and b is unemployment benefits.

The stochastic matrix governing employment, unemployment transitions is given by:

$$\begin{pmatrix} 1 - M_t & \delta \\ M_t & 1 - \delta \end{pmatrix} \tag{99}$$

Again M_t is governed by the labor market equations above.

The firm solves the Dixit-Stiglitz cost problem:

$$y_{j,t} = \left(\frac{p_{j,t}}{P_t}\right)^{-\varepsilon} Y_t \tag{100}$$

with price index P_t given by

$$P_t^{1-\varepsilon} = \int_0^1 p_{j,t}^{1-\varepsilon} dj. \tag{101}$$

The Calvo pricing adjustment parameter θ implies that the intermediate firm chooses price p^* to maximize:

$$\mathbb{E}_t \sum_{s=t}^{\infty} \theta^{s-t} R_{t,s}^{-1} \left[\frac{p_t^*}{P_s} y_{j,s} - w_s n_{j,s} - \psi M_s h_{j,s}\right] \tag{102}$$

Here $n_{j,t}$ denotes employment at firm j in time t. $h_{j,t}$ denotes hiring and $R_{t,s}^{-1}$ denotes the real interest rate discounted s periods back to time t. This model is solved using Reiter's method in combination with time iteration. This model matches unemployment and interest rate to the same data generated by a parameterized model, only two observed variables are used as the model only had two shocks.

BG. Winberry Approach to Krusell-Smith

The second HANK model is Krusell and Smith (1998) solved with Winberry (2018), which is presented in the results and also the basis for my empirical application. The consumer is standard:

$$E[\sum_t = 0^{\infty} \beta^t U(c_t) \tag{103}$$

Utility function is CRRA:

$$U(c-t) = \frac{c^{1-\sigma} - 1}{1-\sigma} \tag{104}$$

The investment equation is:

$$y = c + k' - (1 - \delta)k \tag{105}$$

Agent can be employed or unemployed. If employed works one unit of labor, else does not work. This is denoted with an ϵ. Employment status transitions via a Markov matrix.

On the market side, wages and rental rate equal their marginal benefit. In particular if aggregating across all capital ($\bar{k}$) and labor ($\bar{l}$):

$$w = (1 - \alpha)z(\bar{k}/\bar{l})^{\alpha} r = \alpha z(\bar{k}/\bar{l})^{1-\alpha} \tag{106}$$

The states are the distribution of capital and labor, denoted Γ, as well as productivity. For each individual, they optimize via:

$$v(k, \epsilon; \Gamma, z) = max_{c,k'}\{U(c) + \beta E[v(k, \epsilon; \Gamma, z)|z, \epsilon]\} \tag{107}$$

subject to

$$c + k' = rk + wl\epsilon + (1 - \delta)k\Gamma' = H(\Gamma, z, z')k' \geq 0 \tag{108}$$

Each agent optimizes according to their own k and ϵ, however the prices are chosen so that when agents are optimizing, the aggregate next period levels of k and ϵ match the levels implied byt the prices. As such you are solving for a fixed point so that given a distribution of k and ϵ, the next period distribution of k and ϵ also correspond to the prices of k and ϵ that would induce agents to acquire those amounts. Winberry takes a different approach. Since you know the individual's law of motion and you can integrate the distribution to get the aggregate prices that generate that law of motion, you can derive a law of motion for the heterogeneous distribution. If you approximate this distribution via moments, you can derive a law of motion for the moments of the distribution. Then you can apply a non heterogeneous agent approaches to solve this model. In particular,

Winberry solves the model in Dynare with perturbation. For more details on the solution method, see Winberry (2018).

Estimating erogenous agent models via a likelihood is difficult if one wants to perform full information data with micro-data as well. The reason is that when evaluating the micro-data one has to calculate the likelihood of the the micro data marginalizing over the distribution of the state. One does this by sampling from the Kalman filter state space, macro variables conditioned on the state, and micro variables conditioned on the macro variables and the state. If you need n points to have good coverage of a distribution, to the best of my knowledge examining the paper, I imagine you will need n^3 samples to have good coverage. Due to the fact that SNPE does not require likelihood calculation, this whole process starting from the Kalman filtering and ending up performing likelihood calculations of many many times, is avoided.

C Background on Simulation Neural Ratio Estimators

CA. General Approach of Simulation-Based Inference

The differences between the two simulation-based inference approaches roughly centers around what conditional density estimator to use. Sequential Neural Ration Estimators (SNRE) uses a GAN and Sequential Neural Posterior Estimators (SNPE) use a normalizing flow. While one can sample a lot of points from the joint, an iterative approach where one fits the density estimator on sampled data then re-samples leads to lower variance. I will start discussing SNRE, by introducing the GAN

CB. Generative Adversarial Networks

While I focus on SNPE, there are related approaches that use a GAN (Goodfellow et al., 2014) as density estimators, Sequential Neural Ratio Estimation (SNRE) and another approach built on SNRE, SNVI. The GAN is an algorithm that uses two competing models moving in tandem to generate data that matches data generated from the real world. Below is a diagram describing the

structure of a basic GAN:

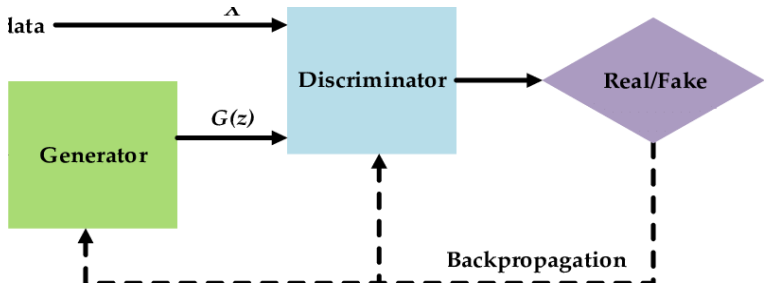

FIGURE XXIX: DIAGRAM OF A GAN

The generator is a neural network (or any differentiable model like a dynamic structural model) which attempts to produce data (unconditionally or conditionally via some inputs Z) that resembles data in the real world. The discriminator is another neural network (or any trainable model that can produce a logistic output), which outputs the probability a given input data comes from the underlying data generating process and not from the generator. The two networks play a game where the objective of the generator is to convince the discriminator to assign high likelihood to its output and the objective of the discriminator is to assign the correct probabilistic label, a number between 0 or 1, to input that represents the continuum that the current data point is generated data or real data respectively. One way to think about a GAN is as a simulated method of moments algorithm where the moment conditions automatically evolve over many rounds to pick the most mismatched moments between the true data and the model. When thought in this way, it is intuitive to recognize that the GAN is nearly a full information and efficient estimation procedure. For more thorough proofs see Kaji, Manresa and Pouliot (2020).

Mathematically the discriminator loss is:

$$Loss_D = -E_{x \sim P_{data}(x)}[D(x)] - E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{109}$$

This is just a logit loss applied to data either coming from the data generating process or the generator. Likewise the generator's loss is to fool the discriminator:

$$Loss_G = E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{110}$$

These two loss functions can be compactly represented as:

$$G, D = \min_G \max_D E_{x \sim P_{data}(x)}[D(x)] + E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{111}$$

Equation (111) makes it clear the GAN objective is a minimax game with a solution that is a Nash equilibrium. However what is important for simulation-based estimation is to recover likelihoods/probabilities. The next proposition following Goodfellow et al. (2014) will show how to do that:

Proposition 1.: For a fixed generator, the optimal discriminator will produce a logistic output:

$$D^*(x) = \frac{P_{data}(x)}{P_{data}(x) + P_{generator}(x)} \tag{112}$$

See Goodfellow et al. (2014) for the proof. Using this, one can use the discriminator output to estimate a likelihood ratio.

CC. Sequential Neural Ratio Estimation

In SNRE, Durkan, Murray and Papamakarios (2020) use a GAN, not to match a underlying data like in proposition 1, but to contrast between two different distributions to estimate a density. In macro, the data set is typically a single data point with correlated time-steps. Using a GAN to discriminate between real data and fake data would not work because the GAN would learn a delta function for the real data. In order to ameliorate this, the authors use contrastive learning where they differentiate between the generated distribution and the independent distribution, $X, \theta \sim P(X)P(\theta)$, where X is sampled uniformly from the empirical distribution of simulated data, rather

than the correct associated θ_i. Then they contrast the independent distribution, with the true joint: $X, \theta \sim p(X, \theta) = p(X|\theta)p(\theta)$. Furthermore, knowing the optimal discriminator gives the ratio discussed in (112). The authors then recover the ratio: $r(X, \theta) = \dfrac{p(X, \theta)}{P(X)P(\theta)}$, using the fact that $\dfrac{p(X, \theta)}{p(X, \theta) + p(X)P(\theta)}$ equals $\dfrac{r}{r+1}$. Thus, the density of the posterior $p(\theta|X) = r(X, \theta)p(\theta)$, where $P(\theta)$ is the prior. This extends Kaji, Manresa and Pouliot (2020) GAN approach in allowing estimation of models with a latent time dimension. One can extend this to multi-round inference where one uses the GAN as a better proposal distribution than the prior, by sampling θ from the GAN using MCMC. Then one uses importance sampling to deal with the mismatch in proposal distribution and prior. See Durkan, Murray and Papamakarios (2020), Hermans, Begy and Louppe (2020) and the SNRE theoretical sections of the appendix for more information.

Algorithm 2: SNRE Algorithm

Input: Simulator $p(X|\theta)$, prior $p(\theta)$, data X', discriminator $d_\phi(X, \theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with MCMC;

  Simulate $X^{(n)} \sim P(X|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{X^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(X, \theta)$ *not converged* **do**

    Sample $\{X^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Sample K contrasting samples for each i sample $\{X^{c(k)}\}_k^{B*K}, \{\theta^{c(k)}\}_k^{B*K} \sim D\{X\}, D\{\theta\}$, breaking the joint distribution of D into independent marginals;

    Train the GAN using the GAN loss function from the two distributions X, θ and X^c, θ^c and a softmax/multinomial logit objective $\sum_i^B log(\frac{exp(d_\phi(X^{(i)}, \theta^{(i)})}{\sum_k^K exp(d_\phi(X^{(k)}, \theta^{(k)})})$;

  end

  Update posterior $p^{(i)} \propto exp(d_\phi(X', \theta))p(\theta)$;

end

CD. Sequential Neural Variational Inference (SNVI)

In SNVI, Glöckler, Deistler and Macke (n.d.), one performs SNRE, but instead of MCMC sampling from the GAN posterior, one projects the distribution derived by the GAN unto a normalizing flow with variational inference. As discussed in this article, MCMC takes a long time to converge to the posterior. Since one can calculate the pdfs of a sample point using the GAN ratio estimation, one can sample from a normalizing flow, calculate the pdf of the sample under both the flow and the GAN, and use this to minimize the KL divergence between the flow and the GAN. Thus the flow inherits the distribution of the GAN, and one can sample directly from the flow without resorting to MCMC. The downside of this approach is that using the reverse KL divergence, in finite iterations,

leads to the learned posterior being too tight around the mode.

Algorithm 3: SNVI Algorithm

Input: Simulator $p(X|\theta)$, prior $p(\theta)$, data X', discriminator $d_\phi(X, \theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim f_{\phi'}^{(i-1)}(\theta)$ for $n = 1...S$;

  Simulate $X^{(n)} \sim P(X|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{X^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(X, \theta)$ *not converged* **do**

    Sample $\{X^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Sample K contrasting samples for each i sample $\{X^{c(k)}\}_k^{B*K}, \{\theta^{c(k)}\}_k^{B*K} \sim D\{X\}, D\{\theta\}$, breaking the joint distribution of D into independent marginals;

    Train the GAN using the GAN loss function from the two distributions X, θ and X^c, θ^c and a softmax/multinomial logit objective $\sum_i^B log(\frac{exp(d_\phi(X^{(i)}, \theta^{(i)})}{\sum_k^K exp(d_\phi(X^{(k)}, \theta^{(k)})})$;

  end

  Train normalizing flow with variational inference $f_{\phi'}^i = argmin_{\phi'}(KL(f_{\phi'}^i(\theta)||exp(d_\phi(X', \theta))p(\theta))$;

end

Department of Economics, University of Michigan, Ann Arbor